

Final project report: GPU Starvation & The Small File Problem — otimizando a ingestão de dados para aceleradores com PySpark ETL + Parquet

Grupo G15 · PDM / Big Data (UFLA) · 2026-1 Gabriel Barros Ferreira · Davi G. Miranda · Lucas Expedito

1. Context and motivation

Um pipeline de Machine Learning para visão computacional é, na prática, um sistema **produtor-consumidor**:

- **Produtor** = CPU + disco: lê o arquivo, **decodifica** (JPEG → pixels), redimensiona, normaliza e entrega o tensor pronto.
- **Consumidor** = GPU: executa o forward/backward do modelo.

Quando o produtor não acompanha o consumidor, a GPU termina cada lote e **fica ociosa esperando o próximo** — o fenômeno de **GPU Starvation** (inanição da GPU). Como a GPU é o recurso caro (US\$ 1–4/h na nuvem, ou custo de oportunidade de ficar parada), mantê-la faminta é desperdício direto de dinheiro.

Esse gargalo se agrava com o **Small File Problem**, um anti-padrão clássico de Big Data: ler **milhares de arquivos .jpg pequenos e independentes**, cada um exigindo abertura, descompressão (decode JPEG) e resize em tempo real — um custo de CPU que não escala.

Objetivo do projeto: provar, com **evidência estatística** (3 rodadas, média ± desvio padrão), que o Small File Problem estrangula a ingestão de dados num pipeline de ML e causa GPU Starvation, e que **converter as imagens para blocos colunares .parquet via um ETL Apache Spark** elimina esse gargalo, aumentando o **throughput de ingestão (imagens/segundo)**.

Tese central: *a forma como os dados são armazenados pode importar tanto quanto o poder de cálculo bruto*. Trocamos disco barato (formato pré-processado) por eficiência de compute (CPU liberada, GPU alimentada).

A jornada científica (honestidade metodológica). A hipótese literal inicial era "small file → gargalo de disco". Testes de teto controlados a **falsificaram**: a leitura pura do SSD entrega **34.615 img/s** com só 24% de CPU — o disco tem ~30× de folga. O gargalo real é o **decode JPEG na CPU**. Isso redirecionou o experimento para o gargalo verdadeiro (CPU-bound), em vez de forçar uma narrativa falsa. Dois pivots reforçaram o rigor: (1) corrigir a métrica de CPU dentro do Docker — **psutil** enxergava os 12 threads do host e escondia a saturação dos 2 vCPUs; passamos a ler a cota via cgroup (`/sys/fs/cgroup/cpu.max`); (2) migrar de **treino** (ResNet50, cujo backward mascarava o gargalo) para **inferência em batch** (MobileNetV2, forward-only), que é o cenário de produção onde o Small File Problem realmente dói.

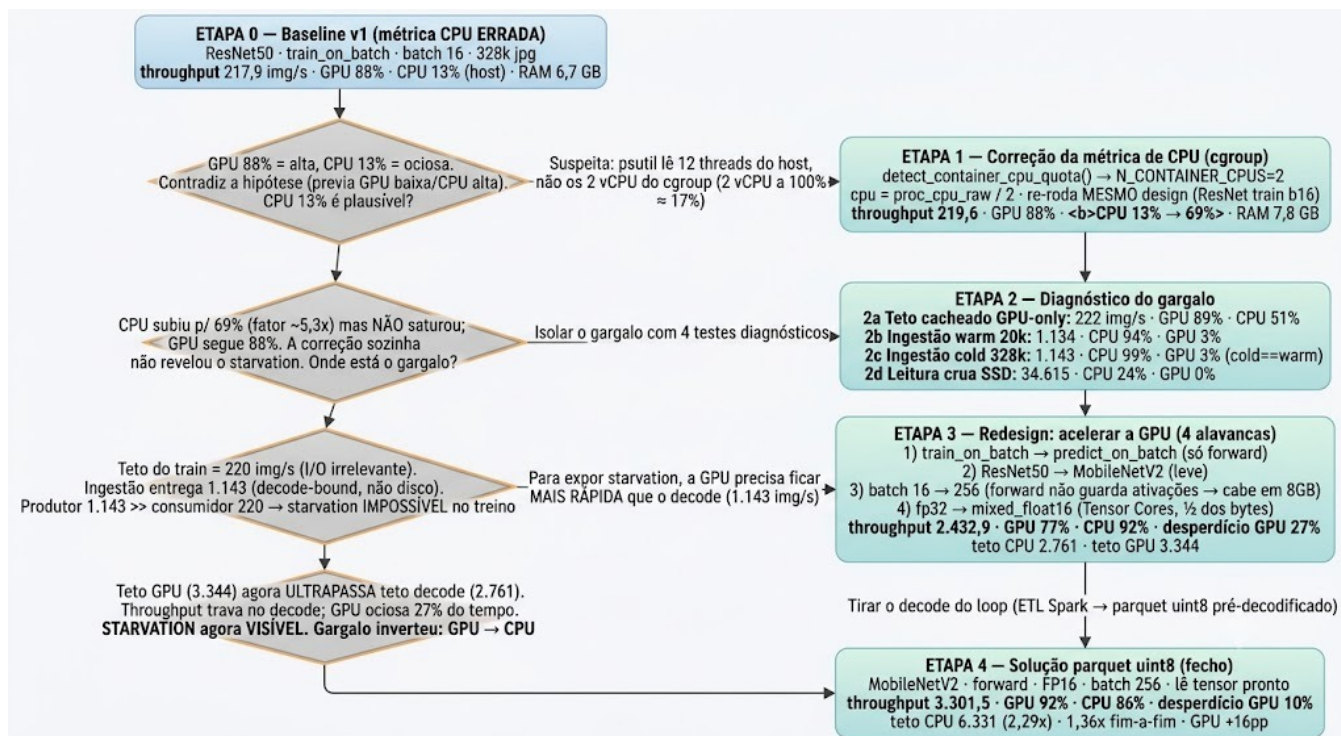


Figura — Cronologia das decisões experimentais, com os números reais que dispararam cada mudança (baseline v1 → correção da métrica de CPU → diagnóstico do gargalo → redesign para inferência → solução Parquet).

2. Data

2.1 Detailed description

- **Fonte:** [Dogs vs. Cats \(Kaggle\)](#). Download manual (Kaggle exige login).
- **Conteúdo bruto:** 25.000 imagens `.jpg` coloridas (RGB), dimensões variadas, ~0,53 GB. O **rótulo** está no nome do arquivo (`cat.*.jpg` → 0, `dog.*.jpg` → 1) — classificação binária.
- **Inflação reprodutível:** as imagens são **duplicadas de forma controlada** (sufixo `_dupN`, preservando o rótulo cat/dog) até **70.433 imagens / ~1,5 GB**. Isso ultrapassa o piso da disciplina sem custo de download e estressa o I/O de forma realista.
- **Transformação (ETL):** durante a conversão, cada imagem é decodificada, redimensionada para o tensor `(224, 224, 3)` e serializada estritamente como `uint8` (0–255, 150.528 bytes por imagem). O `.parquet` resultante ocupa **~9,4 GB em ~35 blocos** — o dado descomprimido explode ~6× e **não cabe confortavelmente em RAM**, forçando leitura de disco (característica de Big Data por Volume).

Por que `uint8` e não bytes JPEG nem `float16`? Guardar bytes JPEG faria o Parquet re-decodificar na leitura → mesmo teto → empate. Guardar `float16` pré-normalizado dobraria o disco (~16,5 GB) e arredondaria o pixel. Guardar o **array já decodificado como `uint8`** ataca o gargalo real (o decode sai do caminho crítico), usa metade do disco e do barramento CPU → GPU, e é **sem perda** (o pixel que sai do decode já é `uint8`). A normalização (`x/127.5 - 1`) migra para a **GPU** no read.

2.2 How to obtain the data

- **Amostra (incluída no repo):** `datasample/sample_images/` — **40 imagens** (20 gatos + 20 cães), ~**862 KB** (abaixo do limite de 1 MB do template). Serve para o *smoke test* que prova que o projeto roda.

⚠ **Aviso:** com tão poucas imagens, o dataset cabe na RAM/cache e o Small File Problem **não aparece** — os números do smoke test são **inconclusivos de propósito**. Os resultados válidos do relatório vêm do dataset completo.

- **Dataset completo (não versionado):**

```
bash # 1. Baixe o "train.zip" (login Kaggle necessario): # https://www.kaggle.com/c/dogs-vs-cats/data
# 2. Coloque em: output/data/raw_jpg/train.zip # 3. Descompacte (dentro do container, so Docker necessario): ./bin/download_data.sh
# A duplicacao ate ~1,5 GB e feita automaticamente pela etapa 01 do bin/run_full.sh.
```

3. How to install and run

Único requisito: Docker + Docker Compose com suporte a GPU NVIDIA (nvidia-container-toolkit). Todo o código roda **exclusivamente dentro do container** — o host permanece intocado. Não é preciso instalar Python, Spark ou Java no host.

A imagem base é a **NVIDIA NGC `tensorflow:25.02-tf2-py3`** (TensorFlow otimizado com CUDA 12.8, necessário para a RTX 5060 / Blackwell `sm_120`, que o TF stock não suporta). O container é limitado a **2 vCPUs + 16 GB RAM** — o limite de 2 vCPUs é justamente o que **cria o gargalo** que estudamos.

3.1 Quick start (using sample data in `datasample/`)

Um único comando faz o build e roda o pipeline completo (baseline → ETL → otimizado) sobre a amostra:

```
./bin/run.sh
```

Resultados (CSVs + PNGs) aparecem em `./output/results/`. (Lembre: com 40 imagens os números são inconclusivos — isto só prova que executa.)

3.2 How to run with the full dataset

Reproduz os resultados do relatório (preparo + duplicação até ~1,5 GB + 3 rodadas de cada benchmark):

```
# 1. Obtenha o dataset (ver secao 2.2):  
./bin/download_data.sh  
# 2. Rode o pipeline completo:  
./bin/run_full.sh  
# Ajustes opcionais:  
TARGET_GB=8 N_ROUNDS=3 BATCH_SIZE=256 ./bin/run_full.sh
```

O **mesmo código** serve para o smoke test e para o dataset completo — só mudam as variáveis de ambiente (`RAW_DIR` , `PARQUET_DIR` , `N_ROUNDS` , `BATCH_SIZE` , `TARGET_GB` ...) passadas pelos scripts em `bin/`.

Modo interativo (inspecionar os notebooks originais em `src/notebooks/`):

```
docker compose -f misc/docker-compose.yml up  
# Abra o JupyterLab em http://localhost:8888 (Spark UI em :4040 durante o ETL)  
# Os 4 notebooks estao montados em /tf/src/notebooks
```

4. Project architecture

Tudo roda em **um único container Docker** (reprodutibilidade total): duas vias de ingestão com um ETL no meio.

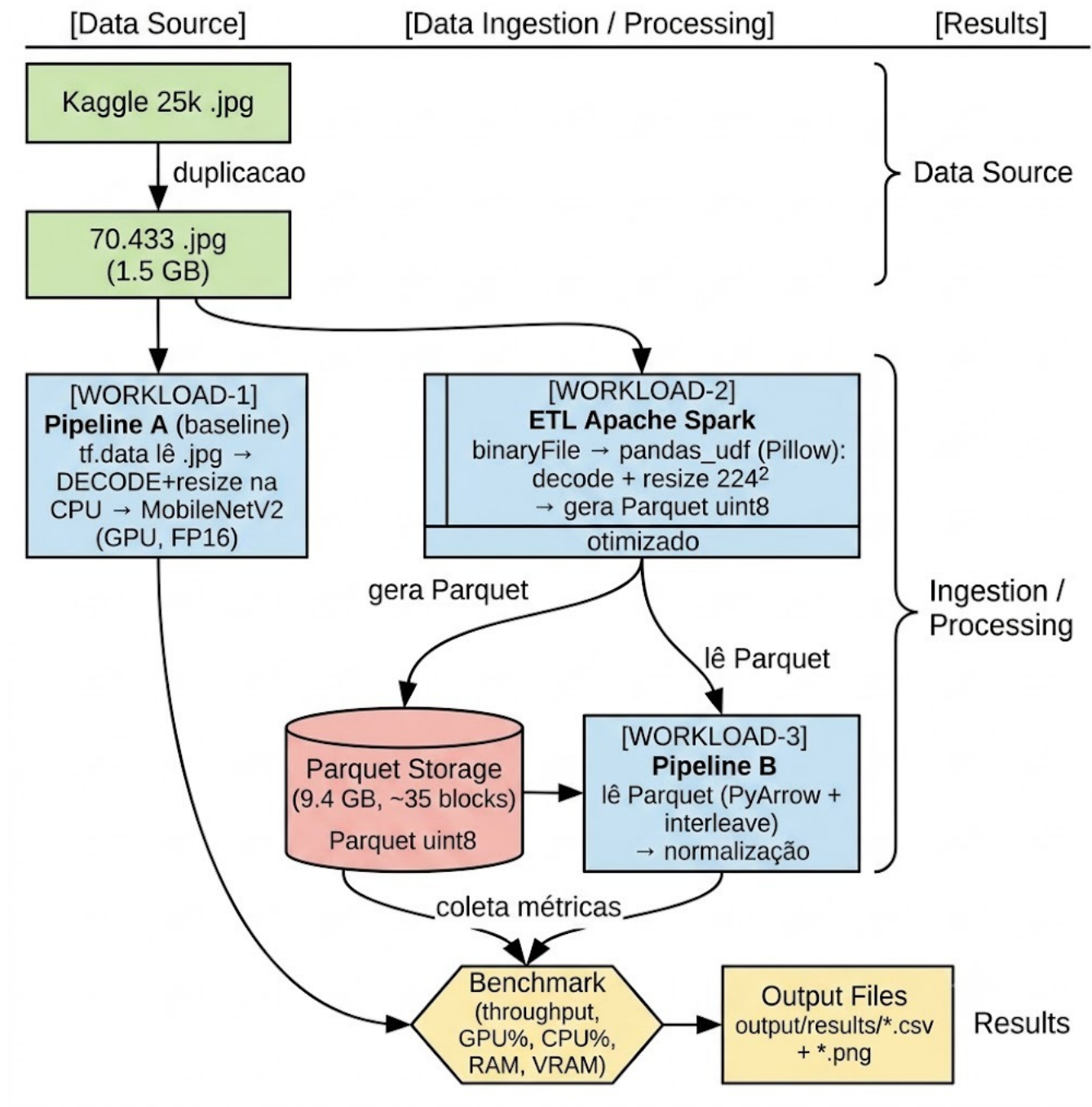
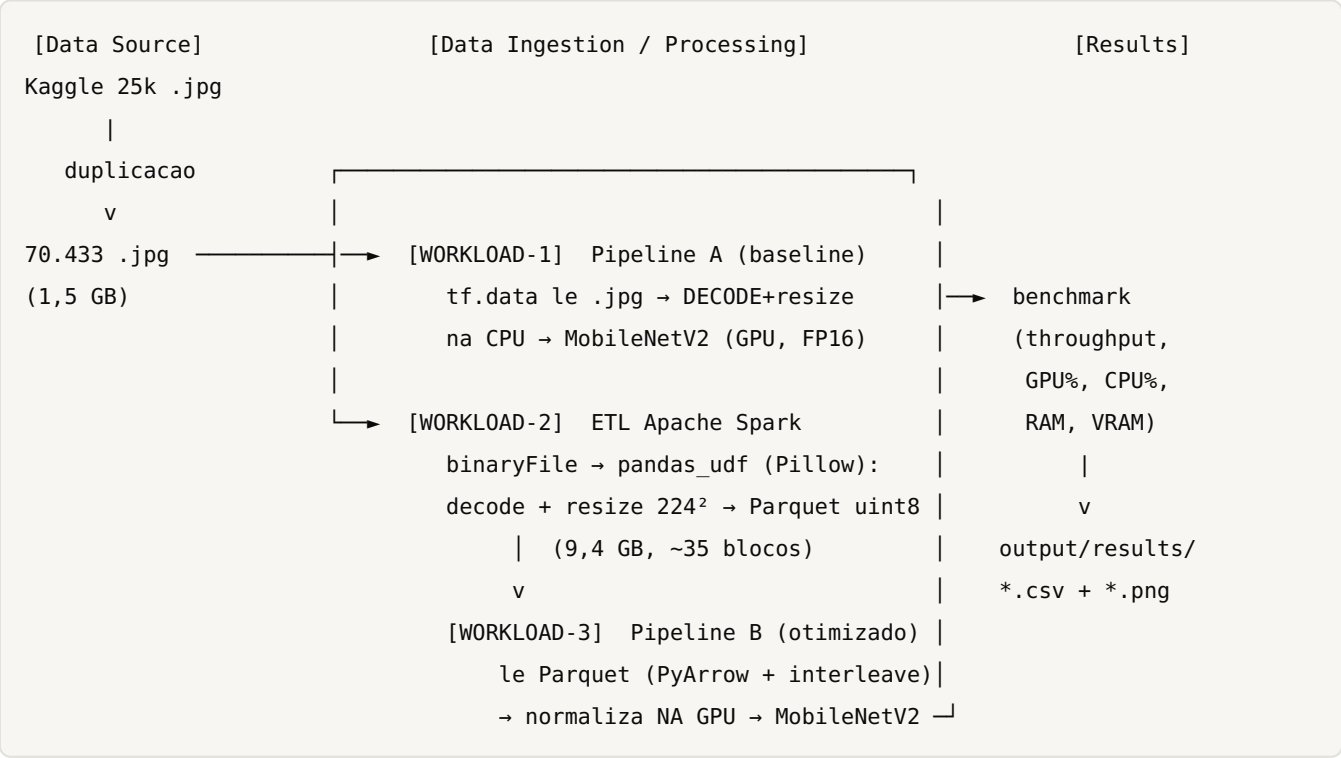


Figura 2 — Arquitetura do pipeline. Diagrama equivalente em texto (ASCII) abaixo.



Componentes e como interagem (todos no mesmo container):

Componente	Stack	Função
Preparo (src/01_download.py)	Python	Move labels do nome do arquivo + duplica até TARGET_GB
Pipeline A (src/02_baseline_jpg.py)	tf.data + MobileNetV2	Ingestão de .jpg soltos (decode na CPU) + benchmark
ETL (src/03_spark_etl.py)	PySpark 3.5 (local[2]) + Pillow	.jpg → .parquet uint8 (offline, pago 1x)
Pipeline B (src/04_optimized_parquet.py)	tf.data + PyArrow + MobileNetV2	Ingestão do .parquet (normaliza na GPU) + comparação
Módulo comum (src/bench_common.py)	psutil + pynvml	Config, monitor de recursos (cgroup-aware), modelo
Orquestrador (src/run_all.py)	Python	Roda 02 → 03 → 04 em processos separados (evita que o cache/estado do TF de uma etapa contamine a medição da outra)

Decisões de engenharia que moldam a arquitetura:

- **Preprocess na GPU (Pipeline B)**: o Parquet entrega uint8 ; o cast+escala tf.cast(x, float16)/127.5 - 1 roda na GPU. A CPU (só 2 vCPUs) fica livre para o que só ela faz (ler o byte e transferir), e trafegamos 1 byte/canal em vez de 4.
- **prefetch(AUTOTUNE)** : sobrepõe produtor e consumidor — enquanto a GPU processa o lote N, a CPU prepara o N+1. É o que torna o gargalo "o mais lento dos dois".
- **interleave(cycle_length=2) (Pipeline B)**: lê vários Parquet em paralelo; cycle_length=2 casa com os 2 vCPUs. O PyArrow libera o GIL no I/O, então paraleliza de verdade.
- **PyArrow, não tensorflow-io** : o tensorflow-io puxaria o TF stock por cima do TF otimizado da NVIDIA → a RTX 5060 pararia de funcionar. Decisão forçada por hardware real.

- **ETL Spark** `local[2]` casando com os 2 vCPUs; `repartition` **ANTES do decode** (~35 blocos) espalha o trabalho pesado e define o número de arquivos de saída, embaralhando só os bytes JPEG leves (~1,7 GB), não os pixels (9,4 GB).
- **Pillow no `pandas_udf` , não TensorFlow:** o TF (~2 GB) não sobrevive ao fork dos workers Spark → segfault/OOM. Pillow é leve e forka limpo; imagem corrompida vira array de zeros (não derruba a task).

5. Workloads evaluated

Comparação **justa**: mesmo modelo (MobileNetV2, ImageNet, forward-only, FP16), mesmo batch (256), mesmo hardware, mesmo `N_IMAGES` (70.433), mesma estrutura de rodadas/tetos. **Só muda a fonte do dado e onde o pré-processamento roda.**

[WORKLOAD-1] Pipeline A — Baseline (ingestão de `.jpg`)

`tf.data` lê os 70.433 arquivos `.jpg` , **decodifica e redimensiona 224×224 na CPU** (`num_parallel_calls=AUTOTUNE`), e alimenta o forward-only do MobileNetV2 (FP16) na GPU. Mede o throughput de ingestão e quantifica o starvation. Cold read garantido por `shuffle=True` + dataset > RAM.

[WORKLOAD-2] ETL Apache Spark (`.jpg` → `.parquet`)

Pré-processa **offline, uma única vez**. Lê os JPEGs como binário (`binaryFile`), num `pandas_udf` (Arrow + Pillow) decodifica + redimensiona + serializa como `uint8` , e grava **Parquet particionado** (`repartition` antes do decode → ~35 blocos grandes). É o investimento único que elimina o Small File Problem.

[WORKLOAD-3] Pipeline B — Otimizado (ingestão de `.parquet`)

`tf.data` lê o Parquet via **PyArrow com `interleave`** (arquivos em paralelo) + `prefetch` , **normaliza na GPU**, mesma inferência MobileNetV2. Mede o throughput após remover o decode do caminho crítico.

6. Experiments and results

Metodologia obrigatória atendida: cada benchmark de ingestão roda **3 rodadas completas** e reporta **média ± desvio padrão amostral (ddof=1)** do throughput, conforme a regra acadêmica do projeto. Os **CSVs brutos** das 3 rodadas (dados por rodada, amostras de recursos ~1×/s e estatísticas agregadas) estão versionados em `misc/results/` como evidência reproduzível; os gráficos deste relatório (em `assets/`) são gerados a partir desses CSVs.

6.1 Experimental environment

Componente	Especificação
SO (host)	Ubuntu (kernel 7.0), host intocado
CPU (host)	Ryzen 5 (NVMe SSD)
GPU	NVIDIA GeForce RTX 5060, 8 GB VRAM (Blackwell <code>sm_120</code> , CUDA 12.8, driver 580)
Container	Docker Compose, imagem NGC <code>tensorflow:25.02-tf2-py3</code> (TF 2.17)
Limites do container	2 vCPUs (força o gargalo) · 16 GB RAM · GPU via NVIDIA Container Toolkit
ETL	PySpark 3.5.0 sobre OpenJDK 17 · Pillow · Arrow habilitado
Ingestão	tf.data (map/interleave + AUTOTUNE + prefetch) · PyArrow · MobileNetV2 FP16

Por que 2 vCPUs? O host é forte demais — com CPU sobrando, o decode acompanharia a GPU e **não haveria gargalo para estudar**. Limitar a 2 vCPUs cria artificialmente o gargalo de produtor, e **não é arbitrário**: reproduz um cenário real e comum (VMs de nuvem com 1 GPU e poucos cores; nós densos com muitas GPUs disputando CPU). Razão vCPU-por-GPU baixa é o cenário econômico normal quando a GPU é o recurso caro.

6.2 How to perform benchmarking

Métricas capturadas por uma thread `ResourceMonitor` amostrando $\sim 1\times/s$ durante a inferência:

1. **Throughput** (img/s) — métrica base para média e desvio.
2. **Tempo total** da rodada (s).
3. **Ocupação média da GPU** (%) via `pynvml / nvidia-ml-py`.
4. **Pico de RAM** (GB) e **CPU** (%) via `psutil` — **normalizados pela cota do cgroup do container** (senão a saturação dos 2 vCPUs fica invisível).
5. **Pico de VRAM** (GB) — com `set_memory_growth(True)` para o `pynvml` medir a VRAM real.

Robustez: **warmup fora do cronômetro** (traça o `@tf.function` e esquento o pipeline, para a rodada 1 não inflar o desvio); orquestração em processos separados; 3 rodadas → média $\pm$ desvio.

6.3 What did you test?

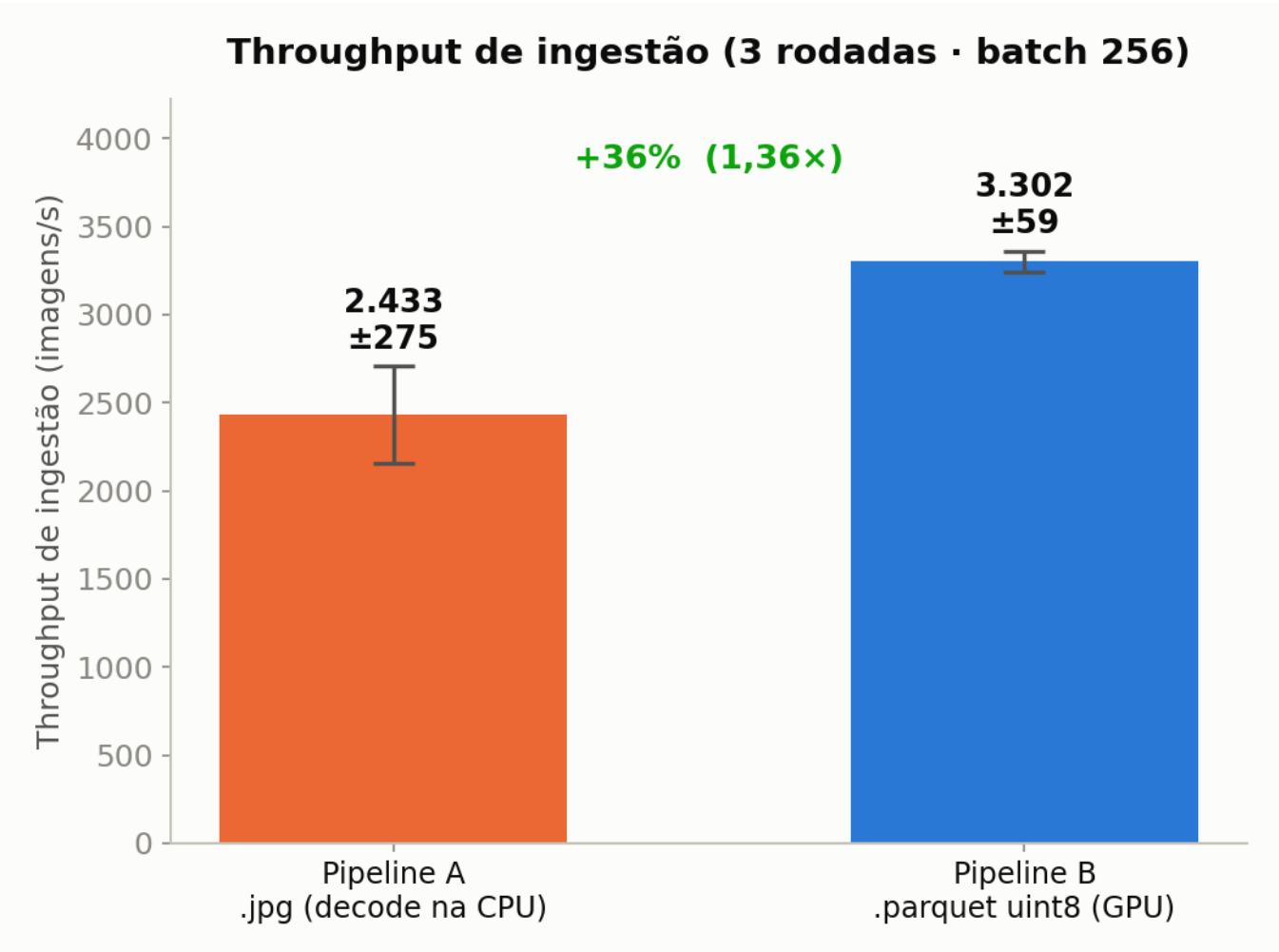
- **Parâmetro variado:** a fonte do dado e o local do pré-processamento — `.jpg` com decode na CPU (Pipeline A) vs `.parquet` uint8 com normalização na GPU (Pipeline B). Todo o resto é mantido constante (comparação justa).
- **Métricas medidas:** throughput (img/s), tempo de rodada, utilização de GPU e CPU, pico de RAM/VRAM, e **tetos de produtor/consumidor** (teto de decode/leitura na CPU vs teto de forward na GPU) — a evidência direta do starvation.
- **Repetições:** 3 rodadas por pipeline, média $\pm$ desvio.

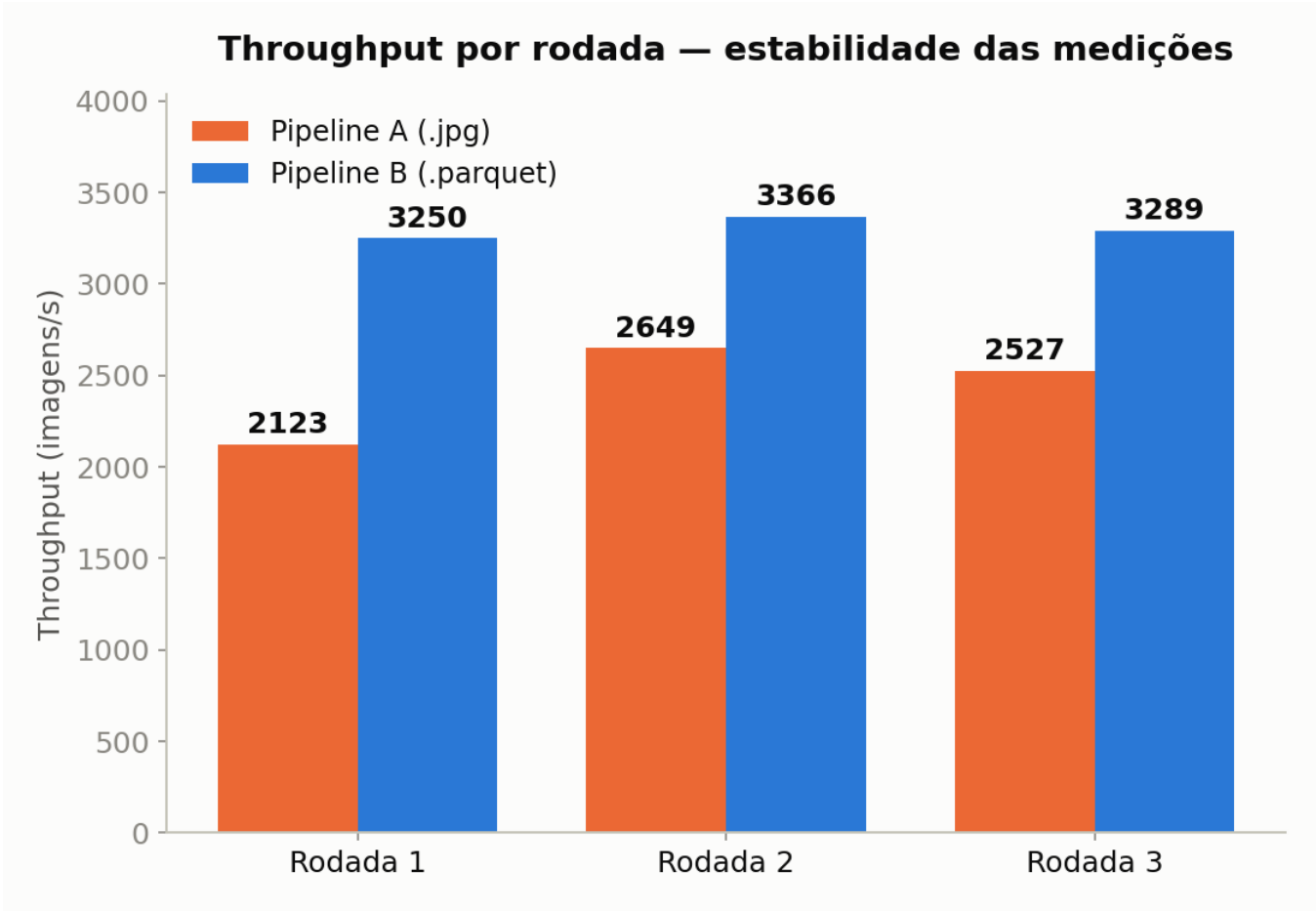
6.4 Results

Métrica principal — Throughput de ingestão (3 rodadas, 70.433 imagens, batch 256)

Workload	Configuration	Avg Throughput (img/s)	Std Dev (img/s)	Runs
WORKLOAD-1 (Pipeline A)	<code>.jpg</code> , decode na CPU	2.432,9	±275,1	3
WORKLOAD-3 (Pipeline B)	<code>.parquet</code> uint8, GPU	3.301,5	±58,9	3

Discussão: o Pipeline B é **+36% (1,36×)** mais rápido **E mais estável** — o desvio caiu de ±275 para ±59 img/s, porque o decode JPEG introduzia variância no baseline. Throughput por rodada (img/s): A = [2.123 · 2.649 · 2.527]; B = [3.250 · 3.366 · 3.289].



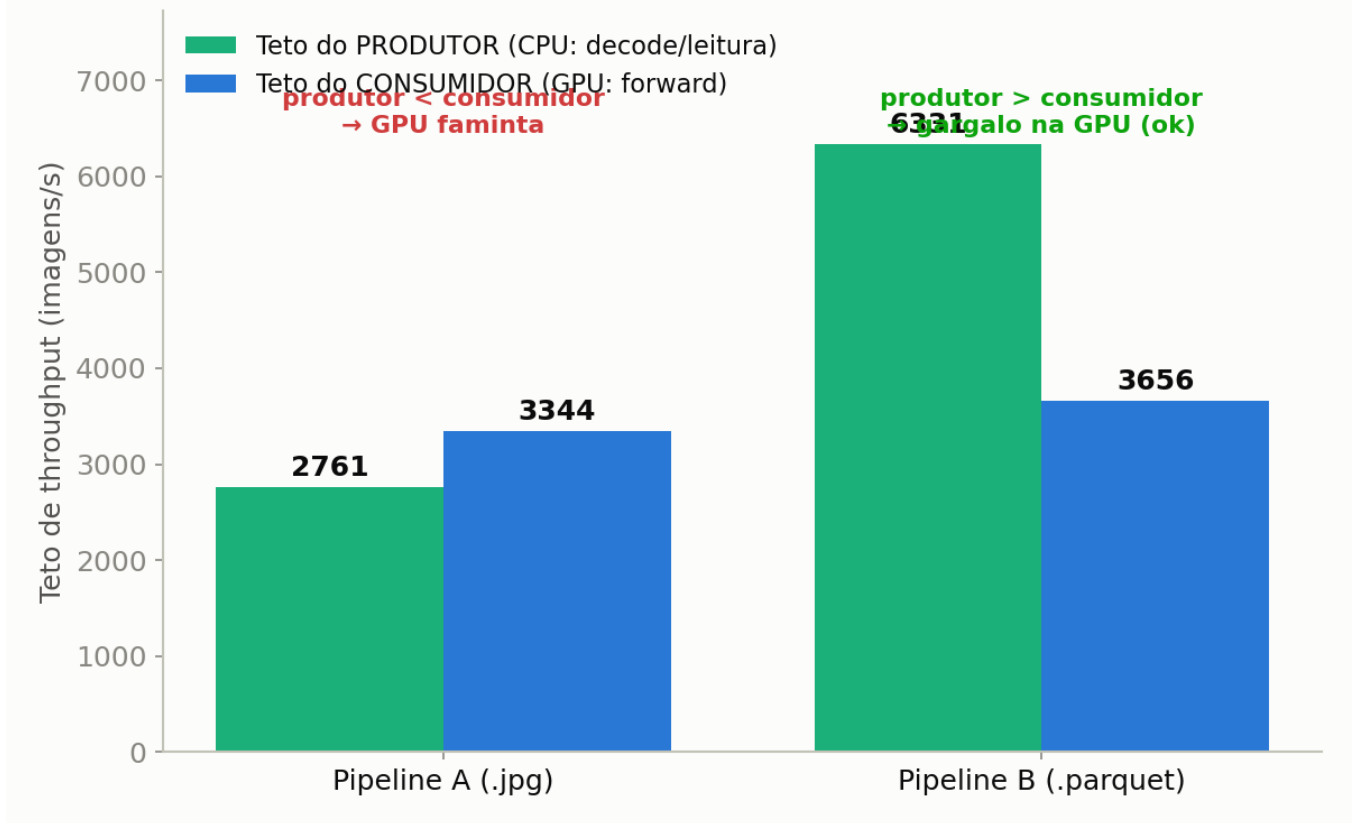


Tetos produtor/consumidor — onde está o gargalo (a prova do starvation)

Cenário	Avg (img/s)	Std Dev (img/s)	Runs
A: teto CPU decode (produtor)	2.760,7	±3,7	3
A: teto GPU forward (consumidor)	3.344,5	±7,6	3
B: teto CPU leitura Parquet (produtor)	6.330,8	±79,7	3
B: teto GPU forward (consumidor)	3.656,1	±9,4	3

Discussão: no baseline o **produtor (2.761) < consumidor (3.344)** → a GPU passa fome. Após o ETL, o produtor fica **2,29× mais rápido (6.331)** e **ultrapassa a GPU** → o gargalo **migra para a GPU** (o limite legítimo do sistema). É a prova de que o decode deixou de ser o gargalo.

Onde está o gargalo: produtor vs consumidor

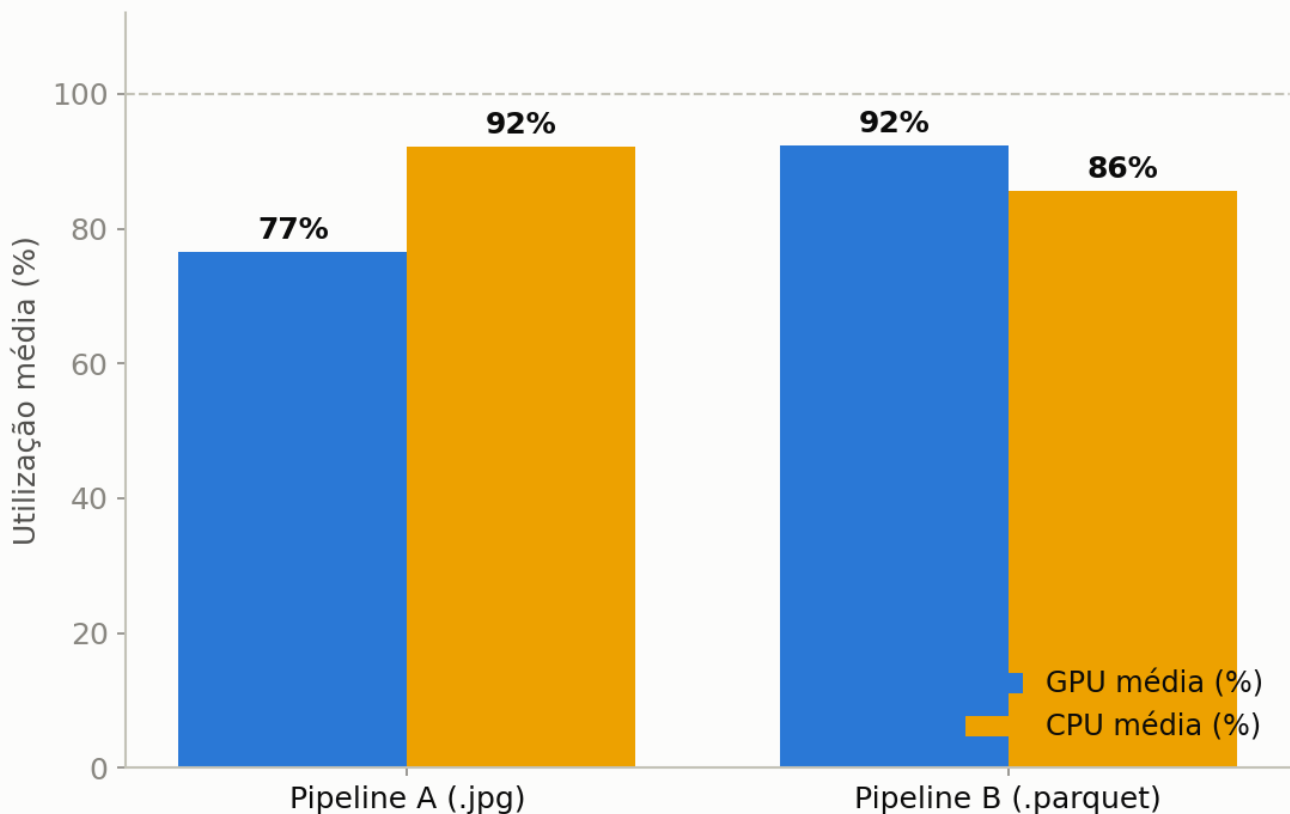


Utilização de recursos e starvation

Configuration	Avg GPU util (%)	Avg CPU util (%)	GPU desperdiçada (starvation, %)	Peak RAM (GB)	Peak VRAM (GB)	Runs
jpg (A)	76,6	92,2	27,3	4,0	4,01	3
parquet (B)	92,4	85,6	9,7	16,0	4,01	3

Discussão: a GPU sobe **77% → 92% (+16 pp)** e o starvation cai **27% → 10% (-18 pp)**. A CPU deixa de estar saturada em decode (92% → 86%): agora só lê bytes prontos. Os ~10% residuais são contenção dos 2 vCPUs entre leitura e dispatch (esperado). O baseline satura a CPU em 100% (máximos por rodada), confirmando o gargalo CPU-bound.

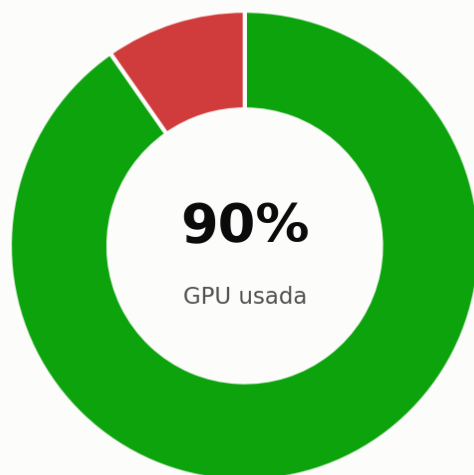
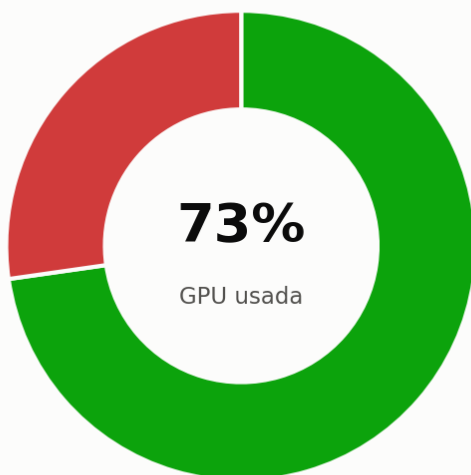
Utilização de recursos (média)



GPU Starvation — fração da GPU ociosa esperando dados

Pipeline A (.jpg)
starvation = 27%

Pipeline B (.parquet)
starvation = 10%



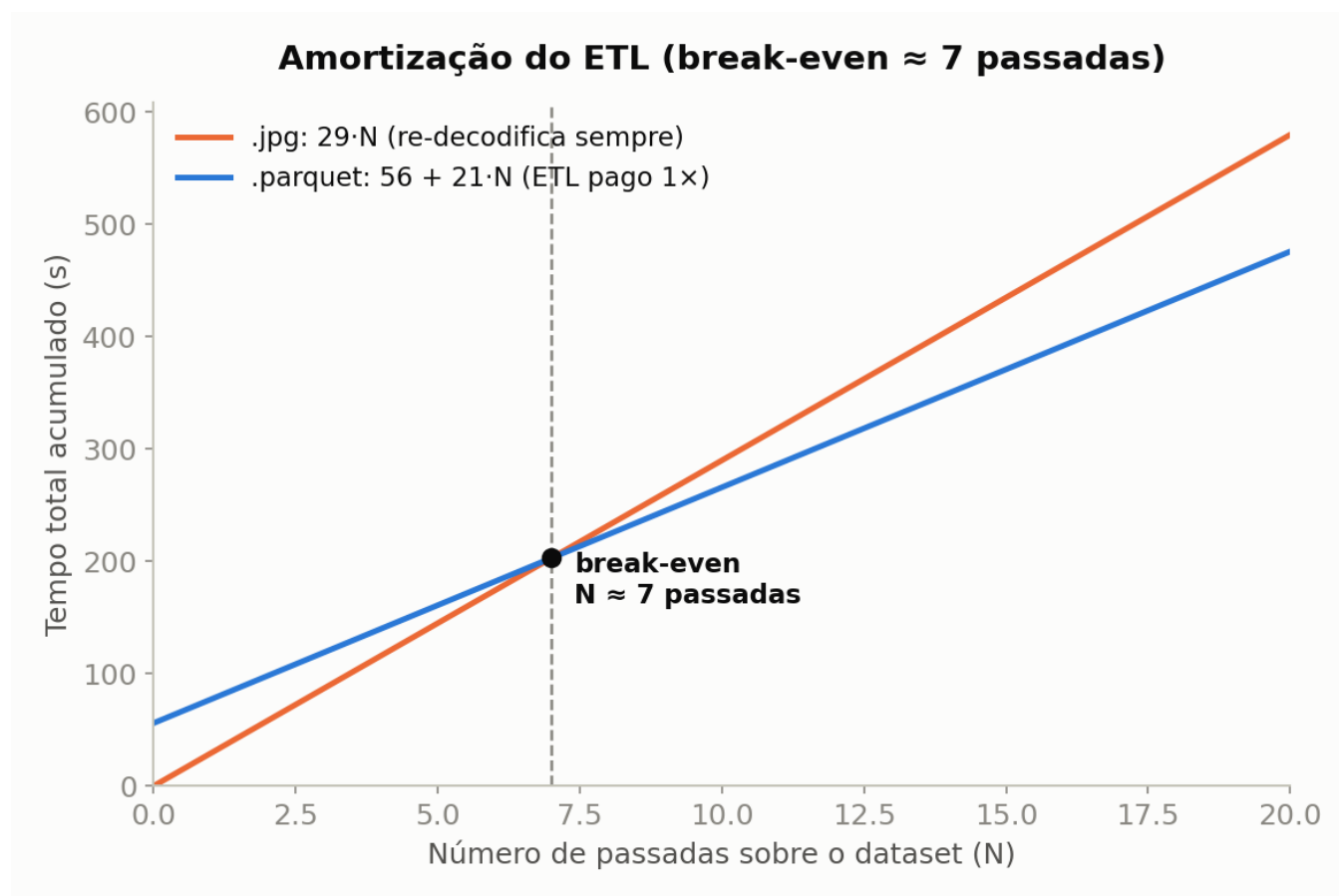
GPU usada GPU ociosa (starvation)

Custo e amortização do ETL (investimento único)

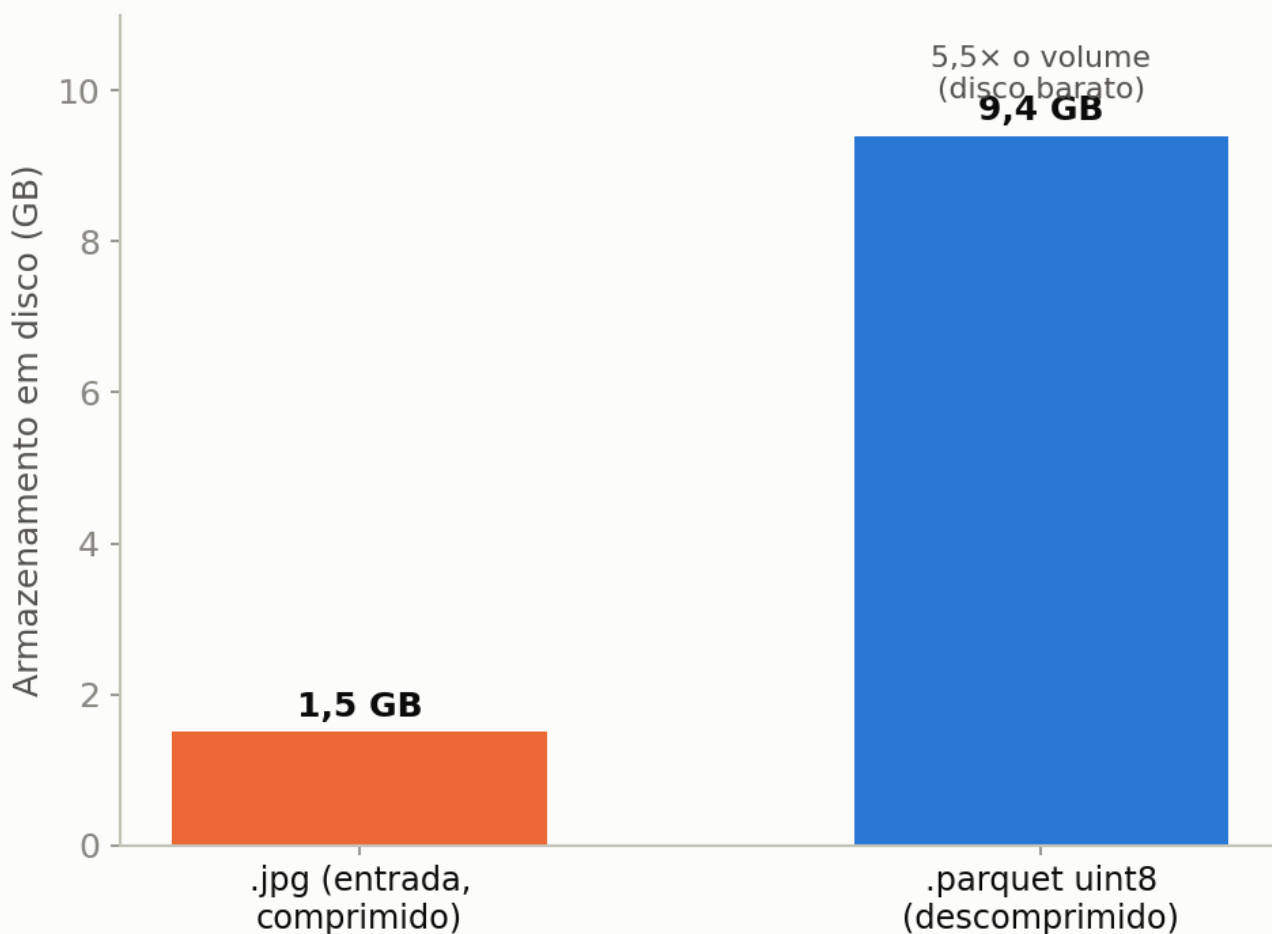
O ETL Spark levou **56 s** para gerar 9,4 GB / 35 Parquet / 70.433 registros. Comparando o tempo total para N passadas de 70k imagens:

.jpg: $29 \cdot N$ s (re-decodifica toda vez, sem ETL)
.parquet: $56 + 21 \cdot N$ s (paga o ETL 1x, depois lê rápido)
break-even: $29 \cdot N = 56 + 21 \cdot N \rightarrow N \approx 7$ passadas

Discussão: a partir de ~7 passadas o Parquet vence no tempo **total**, e cada passada seguinte economiza ~8 s. Treino real roda dezenas a centenas de épocas → o Parquet vence folgado; o único custo remanescente são GB de disco barato (5,5× o volume de entrada).



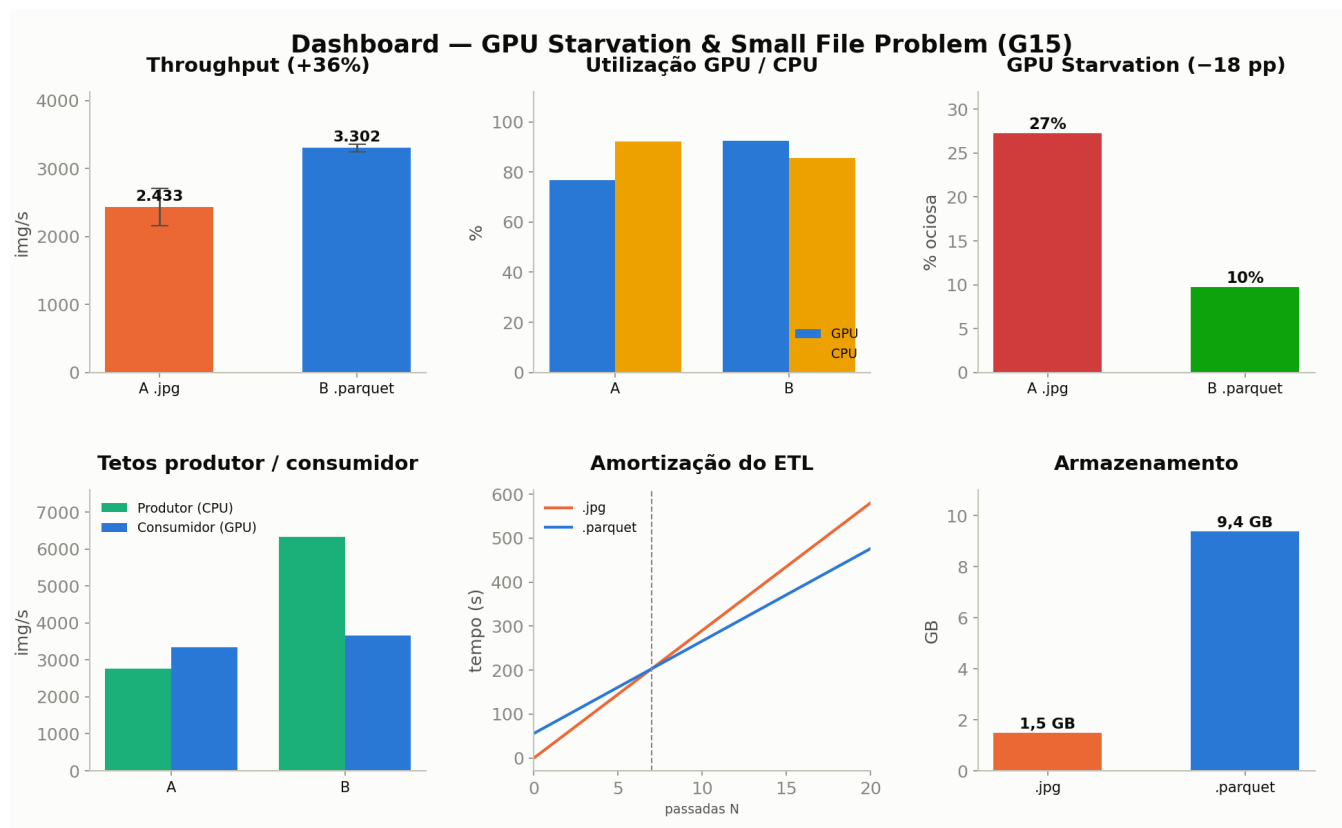
Custo de armazenamento em disco



Resumo (KPIs) e dashboard

Resumo dos KPIs — .jpg (baseline) → .parquet uint8 (otimizado)

+36%	+16 pp	−18 pp	2,29×
Throughput fim-a-fim	Utilização da GPU	GPU Starvation	Teto do produtor (CPU)
2.433 → 3.302 img/s	76,6% → 92,4%	27,3% → 9,7%	2.761 → 6.331 img/s



Interpretação do número "Big Data": a capacidade de ingestão ficou **2,29× maior**. O ganho fim-a-fim é **1,36×** (não 3×) **porque o gargalo mudou de lado**: agora é a GPU (3.656 img/s) que limita, não mais o decode. Esse é o comportamento esperado e correto.

7. Limitations and conclusions

O que funcionou

Provamos experimentalmente, com estatística, que o Small File Problem estrangula a ingestão na CPU e causa GPU Starvation, e que um ETL Spark barato (pago 1×) que reempacota os dados em Parquet **inverte o gargalo**: de um sistema CPU-bound e ocioso (77% GPU, 27% starvation) para um sistema GPU-bound e eficiente (92% GPU, 10% starvation), com **+36% de throughput e maior estabilidade** — sem trocar hardware nem modelo. A jornada (falsificar a hipótese de disco, corrigir a métrica de CPU, migrar treino → inferência) é parte do rigor do trabalho.

Limitações (declaradas)

- **Nó único**: o Spark rodou em modo local (`local[2]`); não avaliamos scale-out multi-nó.
- **ETL medido 1×**: o tempo do ETL (~56 s) foi registrado em uma única execução; para rigor pleno deveria ser repetido $\geq 3\times$ (as ingestões A e B já têm 3 rodadas).
- **RAM entre execuções**: o baseline foi registrado com 4 GB e o otimizado com 16 GB de pico. Como o gargalo do baseline é a **CPU de decode** (não a RAM), isso não muda a conclusão — mas o ideal é re-rodar o baseline com os mesmos limites.
- **Dataset inflado por duplicação**: aumenta volume/I-O de forma realista, mas não a diversidade estatística das imagens (irrelevante para a métrica de throughput, que é o objeto de estudo).

- **Escopo = inferência em batch:** medimos forward-only de propósito, para o custo do backward não esconder o efeito da ingestão. No treino real (dezenas de épocas) o ganho seria **ainda maior** pela amortização do ETL.
- **Quando o trade NÃO vale:** pouquíssimas passadas / inferência isolada (o ETL não amortiza); storage escasso/caro em escala de petabytes; ou CPU/GPU folgado (o ETL nem é necessário).

Conclusão

A forma como os dados são armazenados pode importar tanto quanto o poder de cálculo bruto.

Mover o pré-processamento pesado para um ETL offline e o leve para a GPU alimentou o acelerador com dado sequencial e barato de ler, resolvendo o starvation sem investir em hardware. Trabalhos futuros: comparar com TFRecords/WebDataset; sharding em cluster Spark multi-nó; medir o impacto em treino completo.

8. References and external resources

- **Dataset:** [Dogs vs. Cats — Kaggle](#)
- **Imagem base:** [NVIDIA NGC TensorFlow 25.02-tf2-py3](#)
- **Apache Spark 3.5 (PySpark):** spark.apache.org/docs/3.5.0
- **Apache Parquet:** parquet.apache.org
- **Apache Arrow / PyArrow:** arrow.apache.org
- **tf.data (TensorFlow):** tensorflow.org/guide/data_performance
- **MobileNetV2:** Sandler et al., 2018 — [arXiv:1801.04381](https://arxiv.org/abs/1801.04381)
- **Pillow:** python-pillow.org
- **psutil:** github.com/giampaolo/psutil · **nvidia-ml-py (pynvml):** pypi.org/project/nvidia-ml-py
- **Prática de indústria (mesmo trade):** [TFRecord](#), [WebDataset](#), [NVIDIA DALI](#)

Notebook 1 — Preparo do dataset (download + duplicação)

01 - Preparação & Inflar do Dataset (Dogs vs. Cats)

Objetivo: organizar as imagens do *Dogs vs. Cats* em `/tf/data/raw_jpg/` e ajustar o tamanho do diretório para **~1,5 GB** (acima do mínimo de **1 GB** exigido na disciplina), via cópia iterativa.

Por quê ~1,5 GB (e não 6-8 GB como antes)? Os testes do notebook 02 mostraram que o gargalo da ingestão é o **decode JPEG na CPU**, não o disco — e o decode roda a cada época independente de cache (medimos *cold* == *warm*: 1.143 vs 1.134 img/s). Logo **não precisamos de um dataset maior que a RAM** pra expor o problema: basta cumprir o mínimo de 1 GB e ter imagens suficientes pra rodadas estáveis. Dataset menor também é **reprodutível** — ninguém precisa gerar dezenas de GB pra testar o trabalho.

Roda **dentro do contêiner**. Os caminhos usam `/tf/data` (volume mapeado de `./data`). — confira o Dockerfile e o docker-compose pra mais detalhes.

Pré-requisito: download manual do dataset

O download é **manual** — não usamos a API do Kaggle. Passo a passo:

1. Acesse <https://www.kaggle.com/competitions/dogs-vs-cats/data> logado e **aceite as regras** da competição (botão *Join Competition / I Understand and Accept*), detalhe: pra conseguir se juntar competição será necessário ter um número de telefone associado a sua conta.
2. Baixe o **train.zip** (~543 MB, 25.000 imagens com a nomenclatura: `cat.0.jpg`, `dog.0.jpg` ...).
3. Coloque o **train.zip** em `./data/raw_jpg/` (no host) e descompacte:

```
cd data/raw_jpg && unzip train.zip && rm train.zip
```

Isso cria `./data/raw_jpg/train/` com as 25.000 imagens.

A partir daqui, as células abaixo cuidam de mover de `train/` para `raw_jpg/` e de inflar o dataset.

```
import shutil
import time
from pathlib import Path

# Configuração de caminhos e da meta de tamanho
BASE_DATA = Path("/tf/data") # volume mapeado de ./data
RAW_DIR = Path("/tf/data/raw_jpg") # destino final das imagens .jpg
TRAIN_DIR = Path("/tf/data/raw_jpg/train") # subpasta criada pelo unzip do train.zip

TARGET_GB = 1.5 # meta de tamanho (>= 1 GB exigido pelo professor)

RAW_DIR.mkdir(parents=True, exist_ok=True)

print(f"RAW_DIR = {RAW_DIR}")
print(f"Meta = {TARGET_GB} GB")
```

```
RAW_DIR = /tf/data/raw_jpg
Meta = 1.5 GB
```

```
# Mover as imagens com label(começa com cat ou dog) de raw_jpg/train/ para raw_jpg/
# Se já estiverem direto em RAW_DIR, só reporta e segue.
if TRAIN_DIR.exists():
    labeled = []
    for img in TRAIN_DIR.rglob("*.jpg"): # Procura na pasta e subpastas pela extensão .jpg, confirma se
    o nome começa com cat ou dog e adiciona na lista
        if img.name.lower().startswith(("cat", "dog")):
            labeled.append(img)
    print(f"Encontradas {len(labeled)} imagens com label no dir {TRAIN_DIR}")

    moved = 0
    for img in labeled: #Pra cada img dentro da lista, move para a pasta RAW_DIR, se ainda não existir
    lá e add +1 no contador
        destiny = RAW_DIR / img.name
        if not destiny.exists():
            shutil.move(img, destiny)
            moved += 1
    print(f"{moved} imagens movidas para {RAW_DIR}")

    shutil.rmtree(TRAIN_DIR, ignore_errors=True) # Apaga a pasta train/
    print(f"Subpasta {TRAIN_DIR.name}/ removida.")

else: # Confere se as imagens com label já estão na pasta RAW_DIR, se não estiverem, levanta um erro
    base = [p for p in RAW_DIR.glob("*.jpg") if p.name.lower().startswith(("cat", "dog"))]
    if base:
        print(f"Sem subpasta train/ – {len(base)} imagens já tão em {RAW_DIR}.")
    else:
        raise FileNotFoundError(
            f"Nenhuma imagem encontrada. Coloque o train.zip em {RAW_DIR} e descompacta"
            "(oia a célula de pré-requisito acima e olha o read me).")
)
```

```
Encontradas 25000 imagens com label no dir /tf/data/raw_jpg/train
25000 imagens movidas para /tf/data/raw_jpg
Subpasta train/ removida.
```

Inflate: duplicação iterativa até a meta de tamanho

A função abaixo varre as imagens e as copia, anexando um sufixo `__dupN` ao nome **preservando o prefixo `cat / dog`** — assim os notebooks de benchmark continuam extraindo o label pelo nome do arquivo. A cada N cópias mede o tamanho do diretório e para ao cruzar `TARGET_GB`.

```

def inflate_directory(path, target_gb):
    # Monta uma lista c todos os arquivos .jpg que nao sao duplicados (nao contem __dup no nome)
    base_files = [p for p in path.glob("*.jpg") if "__dup" not in p.name]
    if not base_files:
        raise RuntimeError("Nenhuma imagem-base encontrada. Rode a célula anterior.")

    # Calcula o tamanho atual do diretório e o tamanho alvo em bytes
    current_bytes = sum(f.stat().st_size for f in path.glob("*.jpg"))
    target_bytes = target_gb * (1024 ** 3)

    print(f"Tamanho inicial: {current_bytes / (1024**3):.2f} GB")
    print(f"Base: {len(base_files):,} imagens")
    print(f"Meta: {target_gb} GB")

    # Se o tamanho atual já for maior ou igual ao alvo, retorna
    if current_bytes >= target_bytes:
        print("Meta atingida")
        return

    # Inicia os contadores de cópias e passes, e o tempo inicial
    copies, dup_pass = 0, 1
    t0 = time.time()

    # Loop principal: enquanto o tamanho atual for menor que o alvo, copia os arquivos base para novos
    arquivos com sufixo __dup{pass}
    while current_bytes < target_bytes:
        for arq in base_files:
            nome_final = path / f"{arq.stem}__dup{dup_pass}{arq.suffix}"
            if not nome_final.exists():
                shutil.copy(arq, nome_final)
                current_bytes += arq.stat().st_size # Incrementa pelo tamanho do arquivo original, pq
                # tem o msm tamanho e n precisa buscar no disco
                copies += 1
            if current_bytes >= target_bytes:
                break

        print(f"  Passe {dup_pass:02d} | {copies:,} cópias | "
              f"{current_bytes/(1024**3):.2f} GB | {time.time()-t0:.0f}s")
        dup_pass += 1

    print(f"\n✅ Inflate concluído: {current_bytes/(1024**3):.2f} GB "
          f"em {copies:,} cópias ({dup_pass-1} passes) - {time.time()-t0:.0f}s")

inflate_directory(RAW_DIR, TARGET_GB)

```

Tamanho inicial: 0.53 GB

Base: 25,000 imagens

Meta: 1.5 GB

Passe 01 | 25,000 cópias | 1.07 GB | 1s

Passe 02 | 45,433 cópias | 1.50 GB | 4s

✅ Inflate concluído: 1.50 GB em 45,433 cópias (2 passes) – 4s

```
# Verificando se deu tudo certo, reportando o total de arquivos e tamanho do diretório
files = list(RAW_DIR.glob("*.jpg"))
n_cat = sum(1 for f in files if f.name.lower().startswith("cat"))
n_dog = sum(1 for f in files if f.name.lower().startswith("dog"))
total_gb = sum(f.stat().st_size for f in files) / (1024 ** 3)

print(f"Dir: {RAW_DIR}")
print(f"Total de imagens: {len(files):,}")
print(f"Gatos : {n_cat:,} imgs")
print(f"Cachorros : {n_dog:,} imgs")
print(f"Tamanho: {total_gb:.2f} GB")
print("Exemplos de nomes:", [f.name for f in files[:3]] + [f.name for f in files[-3:]])
```

```
Dir: /tf/data/raw_jpg
Total de imagens: 70,433
Gatos : 35,207 imgs
Cachorros : 35,226 imgs
Tamanho: 1.50 GB
Exemplos de nomes: ['cat.5749.jpg', 'dog.4832.jpg', 'cat.4470__dup1.jpg', 'cat.9920__dup1.jpg',
'cat.8956__dup1.jpg', 'dog.3686.jpg']
```

Notebook 2 — Pipeline A (baseline): ingestão de .jpg

02 - Baseline: Ingestão de .jpg soltos (Pipeline A)

Hipótese: num pipeline de inferência com a GPU otimizada (MobileNetV2 *forward-only* + mixed precision FP16 + batch grande), a GPU consome imagens **mais rápido do que a CPU consegue decodificar os .jpg**. Ler milhares de arquivos soltos e decodificá-los na CPU vira o gargalo, **deixando a GPU ociosa (GPU Starvation)** e derrubando o **Throughput** (imagens/segundo).

O que este notebook faz:

1. Monta um pipeline `tf.data` otimizado (paralelismo + prefetch) lendo `/tf/data/raw_jpg/*.jpg`.
2. Usa uma `MobileNetV2` (pesos ImageNet) em **inferência** (forward-only) + **FP16** como consumidor rápido de GPU.
3. Roda **N_ROUNDS** rodadas medindo, via thread paralela (`psutil` + `nvidia-ml-py`), CPU%, RAM do contêiner (via cgroup) e GPU% a cada ~1s.
4. Calcula **Média ± Desvio Padrão** do Throughput, plota os gráficos e exporta `.csv`.

Este é o **Pipeline A** (ingênuo: `.jpg` soltos). Será comparado com o **Pipeline B** (`.parquet` pré-decodificado via Spark) nos notebooks 03/04.

Roda dentro do contêiner (4GB RAM / 2 vCPUs). Os caminhos usam `/tf/data`.

```

#importante as bibliotecas
import os
import time
import threading
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import psutil
import pynvml
import tensorflow as tf

# Mixed precision (FP16): corta pela metade a VRAM de ativacao (cabe batch maior) e usa os
# tensor cores (mais rapido). DEVE ser configurado ANTES de construir o modelo.
tf.keras.mixed_precision.set_global_policy("mixed_float16")

# Configs iniciais do experimento
DATA_GLOB = "/tf/data/raw_jpg/*.jpg" # Path das imagens soltas
RESULTS_DIR = Path("/tf/data/results") # Saida dos CSVs do experimento
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

IMG_SIZE = (224, 224) # Tamanho de entrada da MobileNetV2
BATCH_SIZE = 256 # Batch grande: forward usa pouca VRAM e amortiza o overhead por passo
N_IMAGES = None # None = TODAS as imagens do dataset; um inteiro = subset (teste rapido)
N_PASSES = 1 # Passadas por rodada. 1 basta com o dataset inteiro; suba se usar subset
N_ROUNDS = 3 # Rodadas (para media e desvio padrao)
SAMPLE_EVERY = 1.0 # Intervalo de amostragem do monitor em s

# Evita que o TF reserve 100% da VRAM de uma vez - aloca conforme necessario (ajuda o pynvml medir)
for gpu in tf.config.list_physical_devices("GPU"):
    try:
        tf.config.experimental.set_memory_growth(gpu, True)
    except Exception as e:
        print("memory_growth:", e)

print("TensorFlow:", tf.__version__)
print("Policy : ", tf.keras.mixed_precision.global_policy().name)
print("GPUs visiveis:", tf.config.list_physical_devices("GPU"))
print("Resultados em:", RESULTS_DIR)

```

```

2026-07-01 01:49:01.587149: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You
may see slightly different numerical results due to floating-point round-off errors from different
computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-07-01 01:49:01.593870: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:485] Unable to
register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been
registered
2026-07-01 01:49:01.601530: E external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:8473] Unable
to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been
registered
2026-07-01 01:49:01.603888: E external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1471] Unable
to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already
been registered
2026-07-01 01:49:01.610209: I tensorflow/core/platform/cpu_feature_guard.cc:211] This TensorFlow
binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX, in other operations, rebuild
TensorFlow with the appropriate compiler flags.

```

```

INFO:tensorflow:Mixed precision compatibility check (mixed_float16): OK
Your GPU will likely run quickly with dtype policy mixed_float16 as it has compute capability of at
least 7.0. Your GPU: NVIDIA GeForce RTX 5060, compute capability 12.0
TensorFlow: 2.17.0
Policy : mixed_float16
GPUs visiveis: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
Resultados em: /tf/data/results

```



```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1782870542.665497    18792 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782870542.694481    18792 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782870542.697861    18792 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782870542.699903    18792 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
```

```
# Inicializa o nvidia-ml-py(pynvml) pra pegar os dados de GPU, se n tiver GPU ou se o pynvml n estiver
instalado, ignora e continua
try:
    pynvml.nvmlInit()
    GPU_HANDLE = pynvml.nvmlDeviceGetHandleByIndex(0)
    gpu_name = pynvml.nvmlDeviceGetName(GPU_HANDLE)
    if isinstance(gpu_name, bytes): # Pega o handle da GPU e o nome, se o nome for bytes, decodifica
pra string
        gpu_name = gpu_name.decode()
    print(f"GPU detectada: {gpu_name}")
except Exception as e:
    GPU_HANDLE = None
    print("pynvml indisponível ou n foi possível achar a GPU, GPU não será monitorada:", e)
```

```
GPU detectada: NVIDIA GeForce RTX 5060
```

```

# file_path e o caminho do arquivo, ex: /tf/data/raw_jpg/cat.1.jpg
def get_label(file_path):
    # Usa TF strings, pq dentro do tf.data o processamento roda em modo grafo
    fname = tf.strings.split(file_path, os.sep)[-1]
    is_cat = tf.strings.regex_full_match(tf.strings.lower(fname), "cat.*")
    return tf.cast(tf.where(is_cat, 0, 1), tf.float32) # 0.0 gato, 1.0 cachorro

def process_path(file_path):
    label = get_label(file_path)
    img = tf.io.read_file(file_path) # le os bytes do disco
    img = tf.io.decode_jpeg(img, channels=3) # DECODE na CPU -> o gargalo mora aqui
    img = tf.image.resize(img, IMG_SIZE) # resize na CPU
    img = tf.keras.applications.mobilenet_v2.preprocess_input(img) # normaliza p/ MobileNetV2 ([-1,
1])
    return img, label

def build_dataset(batch_size):
    # from_tensor_slices sobre ALL_FILES; shuffle espalha a leitura pelo disco
    ds = tf.data.Dataset.from_tensor_slices(ALL_FILES)
    ds = ds.shuffle(len(ALL_FILES), reshuffle_each_iteration=True)
    ds = ds.map(process_path, num_parallel_calls=tf.data.AUTOTUNE)
    ds = ds.batch(batch_size)
    ds = ds.prefetch(tf.data.AUTOTUNE) # sobrepo e decode (CPU) e inferencia
(GPU)
    return ds

# N_IMAGES=None -> usa TODAS as imagens do dataset; senao um subset (teste rapido). O tamanho nao
# afeta as TAXAS medidas (gargalo e o decode por imagem), so a duracao/estabilidade das rodadas.
ALL_FILES = tf.io.gfile.glob(DATA_GLOB)
if N_IMAGES is not None:
    ALL_FILES = ALL_FILES[:N_IMAGES]
if len(ALL_FILES) == 0:
    raise FileNotFoundError(f"Nenhum .jpg em {DATA_GLOB}. Rode o 01_download.ipynb antes.")
print(f"Imagens no dataset: {len(ALL_FILES):,} (N_IMAGES={N_IMAGES})")

```

Imagens no dataset: 70,433 (N_IMAGES=None)

```
# Consumidor de GPU: MobileNetV2 (ImageNet) em INFERENCIA (forward-only) + FP16.
# Modelo leve: consome imagens MUITO mais rapido que a CPU decodifica -> o gargalo cai
# na ingestao (.jpg) e a GPU aparece faminta (starvation bem mais acentuado que a ResNet50).
def build_model():
    base = tf.keras.applications.MobileNetV2(
        weights="imagenet", include_top=False,
        input_shape=(*IMG_SIZE, 3), pooling="avg",
    )
    # dtype="float32" na saida: boa pratica de mixed precision (estabilidade numerica)
    out = tf.keras.layers.Dense(1, activation="sigmoid", dtype="float32")(base.output)
    return tf.keras.Model(base.input, out)

model = build_model()

@tf.function
def infer(x):
    return model(x, training=False)      # forward-only, compilado em grafo (caminho rapido)

print("Modelo pronto (MobileNetV2, inferencia + FP16). Parametros:", f"{model.count_params():,}")
```

```
I0000 00:00:1782870542.864744 18792 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782870542.868368 18792 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782870542.869550 18792 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782870542.944679 18792 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782870542.945642 18792 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782870542.946417 18792 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
2026-07-01 01:49:02.947168: I tensorflow/core/common_runtime/gpu/gpu_device.cc:2021] Created
device /job:localhost/replica:0/task:0/device:GPU:0 with 2920 MB memory: -> device: 0, name: NVIDIA
GeForce RTX 5060, pci bus id: 0000:01:00.0, compute capability: 12.0
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/
mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 [=====] - 0s 0us/step
Modelo pronto (MobileNetV2, inferencia + FP16). Parametros: 2,259,265
```

```

# -----
# Thread de monitoramento: amostra CPU%, RAM e GPU% a cada SAMPLE_EVERY s.
# -----
# Dentro do Docker, o psutil le /proc do HOST -> nao respeita os limites do cgroup.
# Corrigimos DUAS metricas lendo o cgroup direto:
# - CPU: normalizada pelos vCPUs do container (senao 2 vCPUs a 100% viram ~17%).
# - RAM: memory.current do container (senao psutil reporta a RAM do HOST inteiro,
#       ex.: 7.8 GB num container limitado a 4 GB).

import os

def detect_container_cpu_quota():
    """Numero de vCPUs disponiveis ao container (float)."""
    # cgroup v2
    try:
        with open("/sys/fs/cgroup/cpu.max") as f:
            quota_str, period_str = f.read().strip().split()
            if quota_str != "max":
                return float(quota_str) / float(period_str)
    except (FileNotFoundError, OSError, ValueError):
        pass
    # cgroup v1
    try:
        with open("/sys/fs/cgroup/cpu/cpu.cfs_quota_us") as f:
            quota = int(f.read().strip())
        with open("/sys/fs/cgroup/cpu/cpu.cfs_period_us") as f:
            period = int(f.read().strip())
        if quota > 0:
            return quota / period
    except (FileNotFoundError, OSError, ValueError):
        pass
    # Fallback: CPUs visiveis ao processo (cpuset) ou logicos
    try:
        return float(len(os.sched_getaffinity(0)))
    except (AttributeError, OSError):
        return float(os.cpu_count() or 1)

def read_container_mem():
    """(usado_gb, limite_gb) da memoria do CONTAINER via cgroup (inclui page cache)."""
    # cgroup v2
    try:
        with open("/sys/fs/cgroup/memory.current") as f:
            used = int(f.read().strip())
        with open("/sys/fs/cgroup/memory.max") as f:
            lim = f.read().strip()
            limit = None if lim == "max" else int(lim)
        return used / 1024 ** 3, (limit / 1024 ** 3 if limit else float("nan"))
    except (FileNotFoundError, OSError, ValueError):
        pass
    # cgroup v1
    try:
        with open("/sys/fs/cgroup/memory/memory.usage_in_bytes") as f:
            used = int(f.read().strip())
        with open("/sys/fs/cgroup/memory/memory.limit_in_bytes") as f:
            limit = int(f.read().strip())
            limit = limit if limit < (1 << 62) else None
        return used / 1024 ** 3, (limit / 1024 ** 3 if limit else float("nan"))
    except (FileNotFoundError, OSError, ValueError):
        return float("nan"), float("nan")

N_HOST_CPUS = psutil.cpu_count(logical=True)
N_CONTAINER_CPUS = detect_container_cpu_quota()
_, CONTAINER_MEM_LIMIT_GB = read_container_mem()
print(f"vCPUs do container (cgroup): {N_CONTAINER_CPUS}")
print(f"Limite de RAM (cgroup) : {CONTAINER_MEM_LIMIT_GB:.1f} GB")
print("=> `cpu_pct` = % dos vCPUs do container; `cont_mem_used_gb` = RAM do container.")

class ResourceMonitor(threading.Thread):
    """

```

Captura a cada `interval` s:

```
cpu_pct          % dos vCPUs do CONTEINER (0-100). METRICA PRINCIPAL de CPU: saturada
                  (~100%) durante o pipeline = decode dos .jpg e o gargalo.
cpu_pct_host      % medio do HOST (12 threads). So referencia/debug.
proc_cpu_raw      % do processo sobre 1 vCPU (pode > 100).
cont_mem_used_gb  RAM do CONTEINER via cgroup (METRICA PRINCIPAL de RAM; inclui page cache).
proc_rss_gb       RSS do processo Python (working set).
ram_used_host_gb  RAM usada no HOST inteiro. So referencia; NAO usar como prova.
gpu_util_pct      Utilizacao da GPU (pynvml). BAIXA durante o pipeline = GPU faminta.
gpu_mem_gb        VRAM ocupada.
"""

def __init__(self, gpu_handle, interval=1.0,
              n_container_cpus=N_CONTAINER_CPUS):
    super().__init__(daemon=True)
    self.handle = gpu_handle
    self.interval = interval
    self.n_container_cpus = max(float(n_container_cpus), 1e-6) # evita divisao por zero
    self._stop_event = threading.Event()
    self.samples = []
    self.proc = psutil.Process()

def run(self):
    psutil.cpu_percent(None) # prime: 1a leitura e descartavel
    self.proc.cpu_percent(None)
    t_start = time.time()
    while not self._stop_event.is_set():
        cpu_host = psutil.cpu_percent(None)
        proc_cpu_raw = self.proc.cpu_percent(None)
        cpu_container = min(proc_cpu_raw / self.n_container_cpus, 100.0)

        cont_used_gb, _ = read_container_mem()
        proc_rss_gb = self.proc.memory_info().rss / (1024 ** 3)
        ram_used_host_gb = psutil.virtual_memory().used / (1024 ** 3)

        gpu_util, gpu_mem_gb = np.nan, np.nan
        if self.handle is not None:
            try:
                gpu_util = float(pynvml.nvmlDeviceGetUtilizationRates(self.handle).gpu)
                gpu_mem_gb = pynvml.nvmlDeviceGetMemoryInfo(self.handle).used / (1024 ** 3)
            except Exception:
                pass

        self.samples.append({
            "t_rel": time.time() - t_start,
            "cpu_pct": cpu_container, # PRINCIPAL (escala do container)
            "cpu_pct_host": cpu_host, # referencia (escala do host)
            "proc_cpu_raw": proc_cpu_raw, # bruto do processo (pode > 100)
            "cont_mem_used_gb": cont_used_gb, # PRINCIPAL (RAM do container via cgroup)
            "proc_rss_gb": proc_rss_gb, # working set do processo
            "ram_used_host_gb": ram_used_host_gb, # referencia (RAM do host inteiro)
            "gpu_util_pct": gpu_util,
            "gpu_mem_gb": gpu_mem_gb,
        })
        self._stop_event.wait(self.interval)

def stop(self):
    self._stop_event.set()
    self.join()
    return pd.DataFrame(self.samples)
```

vCPUs do container (cgroup): 2.0

Limite de RAM (cgroup) : 4.0 GB

=> `cpu\_pct` = % dos vCPUs do container; `cont\_mem\_used\_gb` = RAM do container.

```

round_summaries = [] # 1 linha por rodada (metricas agregadas)
raw_samples = []     # serie temporal de todas as amostras (p/ graficos)

# Warmup (FORA do cronometro): traca o @tf.function + esquentando o pipeline/disco, pra a
# rodada 1 nao carregar o custo do 1o tracado + leitura fria e inflar o desvio padrao.
_ds_warm = build_dataset(BATCH_SIZE)
for x, _ in _ds_warm.take(5):
    infer(x)

for r in range(1, N_ROUNDS + 1):
    print(f"\n===== Rodada {r}/{N_ROUNDS} =====")
    ds = build_dataset(BATCH_SIZE) # dataset (N_IMAGES), shuffle -> decode a cada passada

    monitor = ResourceMonitor(GPU_HANDLE, interval=SAMPLE_EVERY)
    monitor.start()

    n_images = 0
    t0 = time.time()
    for _ in range(N_PASSES):
        # varias passadas -> rodada longa o bastante p/ estatistica
        for x, y in ds:
            # y (label) nao e usado na inferencia
            infer(x) # forward-only na GPU (consumidor rapido)
            n_images += int(x.shape[0])
    elapsed = time.time() - t0

    df_s = monitor.stop()
    df_s["round"] = r
    raw_samples.append(df_s)

    throughput = n_images / elapsed
    summary = {
        "round": r,
        "n_images": n_images,
        "round_time_s": elapsed,
        "throughput_img_s": throughput,
        "cpu_mean_pct": df_s["cpu_pct"].mean(),
        "cpu_max_pct": df_s["cpu_pct"].max(),
        "cont_mem_peak_gb": df_s["cont_mem_used_gb"].max(),
        "proc_rss_peak_gb": df_s["proc_rss_gb"].max(),
        "ram_host_peak_gb": df_s["ram_used_host_gb"].max(),
        "gpu_util_mean_pct": df_s["gpu_util_pct"].mean(),
        "gpu_util_max_pct": df_s["gpu_util_pct"].max(),
        "gpu_mem_peak_gb": df_s["gpu_mem_gb"].max(),
    }
    round_summaries.append(summary)
    print(f"  imagens={n_images:,} | tempo={elapsed:.1f}s | "
          f"throughput={throughput:.1f} img/s | "
          f"CPU medio={summary['cpu_mean_pct']:.0f}% | "
          f"GPU media={summary['gpu_util_mean_pct']:.0f}%")

summary_df = pd.DataFrame(round_summaries)
raw_df = pd.concat(raw_samples, ignore_index=True)
summary_df

```

```

# -----
# TETOS ISOLADOS (img/s): quanto a CPU (decode) e a GPU (forward) aguentam sozinhas.
# -----
# Quantifica o starvation: o pipeline .jpg fica preso perto do teto da CPU (produtor),
# bem abaixo do teto da GPU (consumidor). A diferenca = capacidade de GPU desperdicada.
# N_ROUNDS repeticoes de cada -> media +/- desvio, igual ao pipeline.

# Batch fixo (ja decodificado) p/ o teto da GPU: forward puro, sem I/O/decode/cache.
x_fixed, _ = next(iter(build_dataset(BATCH_SIZE)))
infer(x_fixed) # warmup: traca o grafo (fora do cronometro)
N_STEPS = max(30, 15000 // BATCH_SIZE) # ~15k imgs por rodada no teste da GPU

cpu_ceils, gpu_ceils = [], []
for r in range(1, N_ROUNDS + 1):
    # (a) TETO DA CPU: decode puro, sem GPU (produtor)
    ds_cpu = build_dataset(BATCH_SIZE)
    n, t0 = 0, time.time()
    for _ in range(N_PASSES):
        for x, y in ds_cpu:
            n += int(x.shape[0])
    cpu_ceils.append(n / (time.time() - t0))

    # (b) TETO DA GPU: forward sobre 1 batch fixo (consumidor), sem I/O
    n, t0 = 0, time.time()
    for _ in range(N_STEPS):
        infer(x_fixed)
        n += int(x_fixed.shape[0])
    gpu_ceils.append(n / (time.time() - t0))

    print(f" rodada {r}: teto CPU={cpu_ceils[-1]:.0f} | teto GPU={gpu_ceils[-1]:.0f} img/s")

cpu_ceiling = float(np.mean(cpu_ceils))
cpu_ceiling_std = float(np.std(cpu_ceils, ddof=1)) if N_ROUNDS > 1 else 0.0
gpu_ceiling = float(np.mean(gpu_ceils))
gpu_ceiling_std = float(np.std(gpu_ceils, ddof=1)) if N_ROUNDS > 1 else 0.0

print(f"\nTeto CPU (decode puro) : {cpu_ceiling:.0f} +/- {cpu_ceiling_std:.0f} img/s (produtor)")
print(f"Teto GPU (forward fixo) : {gpu_ceiling:.0f} +/- {gpu_ceiling_std:.0f} img/s (consumidor)")

```

2026-07-01 01:50:58.561800: I tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

```

rodada 1: teto CPU=2760 | teto GPU=3336 img/s
rodada 2: teto CPU=2757 | teto GPU=3348 img/s
rodada 3: teto CPU=2765 | teto GPU=3349 img/s

```

```

Teto CPU (decode puro) : 2761 +/- 4 img/s (produtor)
Teto GPU (forward fixo) : 3344 +/- 8 img/s (consumidor)

```

```

tp = summary_df["throughput_img_s"].values
tp_mean = tp.mean()
tp_std = tp.std(ddof=1) if len(tp) > 1 else 0.0

# Starvation quantificado: % do teto da GPU que o pipeline .jpg NAO consegue aproveitar
gpu_waste_pct = 100.0 * (1 - tp_mean / gpu_ceiling) if gpu_ceiling else float("nan")

stats = {
    "pipeline": "jpg_forward_fp16",
    "n_rounds": N_ROUNDS,
    "batch_size": BATCH_SIZE,
    "n_images_round": int(summary_df["n_images"].iloc[0]),
    "throughput_mean_img_s": tp_mean,
    "throughput_std_img_s": tp_std,
    "cpu_ceiling_img_s": cpu_ceiling,          # teto da CPU (decode puro) = produtor
    "cpu_ceiling_std_img_s": cpu_ceiling_std,
    "gpu_ceiling_img_s": gpu_ceiling,          # teto da GPU (forward fixo) = consumidor
    "gpu_ceiling_std_img_s": gpu_ceiling_std,
    "gpu_waste_pct": gpu_waste_pct,           # starvation: teto da GPU nao aproveitado
    "round_time_mean_s": summary_df["round_time_s"].mean(),
    "cpu_mean_pct": summary_df["cpu_mean_pct"].mean(),
    "gpu_util_mean_pct": summary_df["gpu_util_mean_pct"].mean(),
    "cont_mem_peak_gb": summary_df["cont_mem_peak_gb"].max(),
    "gpu_mem_peak_gb": summary_df["gpu_mem_peak_gb"].max(),
}
stats_df = pd.DataFrame([stats])

summary_df.to_csv(RESULTS_DIR / "jpg_forward_per_round.csv", index=False)
stats_df.to_csv(RESULTS_DIR / "jpg_forward_stats.csv", index=False)
raw_df.to_csv(RESULTS_DIR / "jpg_forward_raw_samples.csv", index=False)

print(f"Teto CPU (decode)      : {cpu_ceiling:.0f} +/- {cpu_ceiling_std:.0f} img/s   (produtor)")
print(f"Teto GPU (forward)      : {gpu_ceiling:.0f} +/- {gpu_ceiling_std:.0f} img/s   (consumidor)")
print(f"Pipeline .jpg (real)      : {tp_mean:.0f} +/- {tp_std:.0f} img/s")
print(f">> GPU DESPERDICADA      : {gpu_waste_pct:.0f}%   (starvation = teto da GPU nao aproveitado pelo .jpg)")
print(f"GPU util media {stats['gpu_util_mean_pct']:.0f}% | CPU media {stats['cpu_mean_pct']:.0f}% | "
      f"RAM pico {stats['cont_mem_peak_gb']:.2f}/{CONTAINER_MEM_LIMIT_GB:.1f} GB")
print(f"CSVs salvos em {RESULTS_DIR}")
stats_df

```

```

Teto CPU (decode)      : 2761 +/- 4 img/s   (produtor)
Teto GPU (forward)      : 3344 +/- 8 img/s   (consumidor)
Pipeline .jpg (real)    : 2433 +/- 275 img/s
>> GPU DESPERDICADA    : 27%   (starvation = teto da GPU nao aproveitado pelo .jpg)
GPU util media 77% | CPU media 92% | RAM pico 4.00/4.0 GB
CSVs salvos em /tf/data/results

```

	pipeline	n_rounds	batch_size	n_images_round	throughput_mean_img_s	throughput_std_img_s
0	jpg_forward_fp16	3	256	70433	2432.889639	275.108086

Gráficos: evidência do *GPU Starvation*

- **Esquerda:** Throughput por rodada com a média $\pm$ desvio.
- **Meio:** utilização da GPU ao longo do tempo — deve ficar **baixa** (vales = GPU faminta esperando o decode dos `.jpg` na CPU).
- **Direita:** CPU médio vs GPU média — **CPU alta + GPU baixa** = o decode dos `.jpg` é o gargalo.

E o gráfico-síntese em **img/s: teto da CPU** (decode) vs **teto da GPU** (forward) vs **pipeline real** — a distância entre o pipeline e o teto da GPU é a **capacidade de GPU desperdiçada** (o tamanho do starvation).


```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

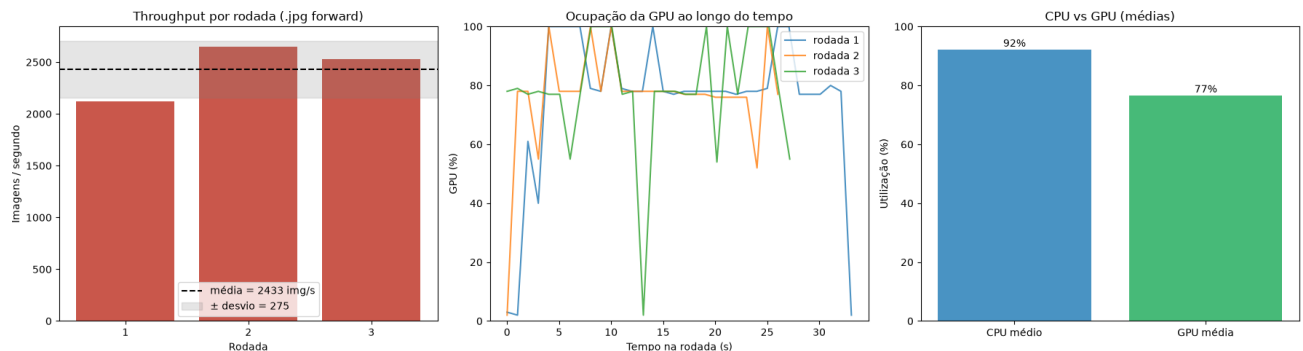
# (1) Throughput por rodada + media/desvio
ax = axes[0]
ax.bar(summary_df["round"].astype(str), summary_df["throughput_img_s"], color="#c0392b", alpha=0.85)
ax.axhline(tp_mean, color="black", ls="--", label=f"média = {tp_mean:.0f} img/s")
ax.axhspan(tp_mean - tp_std, tp_mean + tp_std, color="gray", alpha=0.2, label=f"± desvio = {tp_std:.0f}")
ax.set_title("Throughput por rodada (.jpg forward)")
ax.set_xlabel("Rodada"); ax.set_ylabel("Imagens / segundo"); ax.legend()

# (2) Utilizacao da GPU ao longo do tempo, por rodada
ax = axes[1]
for r, g in raw_df.groupby("round"):
    ax.plot(g["t_rel"], g["gpu_util_pct"], label=f"rodada {r}", alpha=0.8)
ax.set_title("Ocupação da GPU ao longo do tempo")
ax.set_xlabel("Tempo na rodada (s)"); ax.set_ylabel("GPU (%)")
ax.set_ylim(0, 100); ax.legend()

# (3) CPU vs GPU medios (o contraste que prova o starvation)
ax = axes[2]
ax.bar(["CPU médio", "GPU média"],
       [stats["cpu_mean_pct"], stats["gpu_util_mean_pct"]],
       color=["#2980b9", "#27ae60"], alpha=0.85)
ax.set_title("CPU vs GPU (médias)")
ax.set_ylabel("Utilização (%)"); ax.set_ylim(0, 100)
for i, v in enumerate([stats["cpu_mean_pct"], stats["gpu_util_mean_pct"]]):
    ax.text(i, v + 1, f"{v:.0f}%", ha="center")

plt.tight_layout()
plt.savefig(RESULTS_DIR / "jpg_forward_plots.png", dpi=120, bbox_inches="tight")
plt.show()

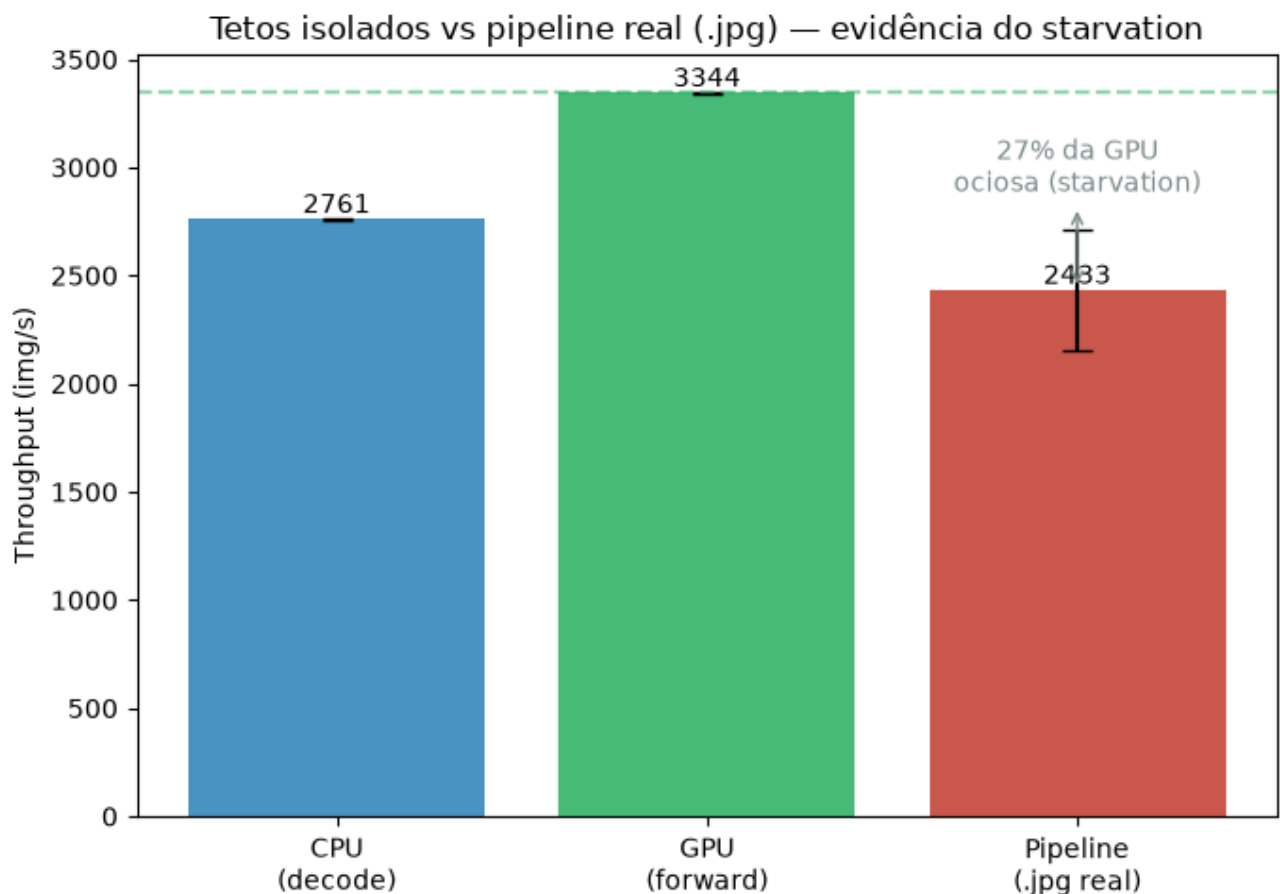
```



```
# Grafico-sintese (img/s): tetos isolados vs pipeline real -> tamanho do starvation
fig, ax = plt.subplots(figsize=(7, 5))
labels = ["CPU\n(decode)", "GPU\n(forward)", "Pipeline\n(.jpg real)"]
vals = [cpu_ceiling, gpu_ceiling, tp_mean]
errs = [cpu_ceiling_std, gpu_ceiling_std, tp_std]
bars = ax.bar(labels, vals, yerr=errs, capsize=6,
              color=["#2980b9", "#27ae60", "#c0392b"], alpha=0.85)
ax.set_ylabel("Throughput (img/s)")
ax.set_title("Tetos isolados vs pipeline real (.jpg) — evidência do starvation")
for b, v in zip(bars, vals):
    ax.text(b.get_x() + b.get_width() / 2, v, f"{v:.0f}", ha="center", va="bottom")

# Seta mostrando a capacidade de GPU desperdicada (entre o pipeline e o teto da GPU)
ax.axhline(gpu_ceiling, color="#27ae60", ls="--", alpha=0.5)
ax.annotate(f"{gpu_waste_pct:.0f}% da GPU\nociosa (starvation)",
            xy=(2, tp_mean), xytext=(2, (tp_mean + gpu_ceiling) / 2),
            ha="center", color="#7f8c8d", fontsize=10,
            arrowprops=dict(arrowstyle="<->", color="#7f8c8d"))

plt.tight_layout()
plt.savefig(RESULTS_DIR / "jpg_forward_ceilings.png", dpi=120, bbox_inches="tight")
plt.show()
```



Notebook 3 — ETL Apache Spark (.jpg → .parquet uint8)

03 - ETL Spark: .jpg -> .parquet (Pipeline B)

Por quê: o notebook 02 diagnosticou que o gargalo é a **CPU decodificando JPEG em tempo real** (decode + resize a cada época), deixando a GPU faminta (~27% ociosa).

Solução (ETL): usar o **Apache Spark** pra decodificar + redimensionar (224×224×3) as imagens **uma única vez** e gravar os pixels prontos em **.parquet particionado**. Na inferência (notebook 04) a CPU só lê o array (sem decode) e o normaliza barato antes de enviar pra GPU.

Formato: **uint8** (pixels crus 0-255), serializado como bytes numa coluna **BinaryType**. É **sem perda** (o pixel que sai do decode já é uint8) e **metade** do float16 (~150 KB/imagem → ~7-8 GB pras 70k). O **preprocess** do MobileNet ($x/127.5 - 1$) fica pro read do nb 04 — custo trivial (um **cast+scale** vetorizado, nada perto do decode que eliminamos). O ETL roda **100% na CPU**.

Roda dentro do contêiner. Saída em `/tf/data/optimized/`. Spark UI em <http://localhost:4040>.

```
import time
from pathlib import Path

import pandas as pd
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.functions import pandas_udf
from pyspark.sql.types import BinaryType

RAW_DIR      = "/tf/data/raw_jpg"      # imagens .jpg de entrada
OUT_DIR      = "/tf/data/optimized"    # parquet de saida (Pipeline B)
IMG_SIZE     = 224                    # dimensao de entrada da MobileNetV2
LIMIT        = None                  # None = TODAS as ~70k (~7-8GB uint8). Um inteiro p/ teste rapido.
IMGS_PER_FILE = 2000                  # alvo de imagens por arquivo parquet

spark = (
    SparkSession.builder
        .appName("etl_jpg_to_parquet_uint8")
        .master("local[2]")                # 2 vCPU do container
        .config("spark.driver.memory", "4g")
        .config("spark.sql.execution.arrow.pyspark.enabled", "true")
        .config("spark.sql.execution.arrow.maxRecordsPerBatch", "512")
    # uint8 e' menor -> batch pode ser maior
        .getOrCreate()
)
spark.sparkContext.setLogLevel("WARN")
print("Spark:", spark.version, "| driver.memory:", spark.conf.get("spark.driver.memory"))
```

Setting default log level to "WARN".
To adjust logging level use `sc.setLogLevel(newLevel)`. For SparkR, use `setLogLevel(newLevel)`.
26/07/01 05:02:51 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform...
using builtin-java classes where applicable

Spark: 3.5.0 | driver.memory: 4g

```
# Le as imagens como binario: colunas path, modificationTime, length, content (bytes).
df = (
    spark.read.format("binaryFile")
        .option("pathGlobFilter", "*.jpg")
        .load(RAW_DIR)
)

# label pelo prefixo do nome do arquivo (cat.* -> 0, dog.* -> 1)
df = df.withColumn("fname", F.element_at(F.split(F.col("path"), "/"), -1))
df = df.withColumn("label", F.when(F.lower(F.col("fname")).startswith("cat"), 0).otherwise(1))

if LIMIT is not None:
    df = df.limit(LIMIT)

n_total = df.count()
n_part = max(2, round(n_total / IMGS_PER_FILE))
print(f"Imagens a processar: {n_total:,} -> {n_part} arquivos parquet")

# repartition ANTES do decode: espalha o trabalho pesado nas tasks E ja define o nº de arquivos
# de saida. Embaralha so os bytes JPEG (~1.7GB), nao os pixels decodificados.
df = df.repartition(n_part)
```

```
[Stage 0:=====>(2185 + 2) / 2202]
```

```
Imagens a processar: 70,433 -> 35 arquivos parquet
```

```
# UDF: JPEG bytes -> PIL decode -> resize 224 -> pixels uint8 (0-255) -> bytes.
# Usa Pillow (leve, seguro no worker Spark). uint8 e' SEM PERDA (o pixel decodificado ja e uint8)
# e metade do float16. O preprocess do MobileNet (x/127.5 - 1) fica pro read do nb 04.
# Imagem corrompida -> array de zeros (fallback, nao derruba a task).
@pandas_udf(BinaryType())
def decode_to_uint8(content: pd.Series) -> pd.Series:
    import io
    import numpy as np
    from PIL import Image

    out = []
    for buf in content:
        try:
            img = Image.open(io.BytesIO(buf)).convert("RGB").resize((224, 224), Image.BILINEAR)
            arr = np.asarray(img, dtype=np.uint8) # (224,224,3) em 0-255, sem perda
            out.append(arr.tobytes())
        except Exception:
            out.append(np.zeros((224, 224, 3), np.uint8).tobytes())
    return pd.Series(out)

# colunas finais: label + data (pixels uint8 224x224x3 serializados em bytes)
df_out = df.select("label", decode_to_uint8(F.col("content")).alias("data"))
```

```
# Escreve o parquet particionado. SEM repartition aqui (ja foi antes do decode) -> n_part arquivos.
t0 = time.time()
df_out.write.mode("overwrite").parquet(OUT_DIR)
etl_secs = time.time() - t0
print(f"ETL concluido em {etl_secs:.0f}s -> {OUT_DIR}")
```

```
[Stage 5:=====> (34 + 1) / 35]
```

```
ETL concluido em 56s -> /tf/data/optimized
```

```
# Validacao: le de volta, confere contagem, reshape/dtype/range de 1 registro e tamanho em disco.
import numpy as np
import subprocess

back = spark.read.parquet(OUT_DIR)
n_out = back.count()
print(f"Registros no parquet : {n_out:,}")
print(f"Schema                : {back.schema.simpleString()}")

row = back.limit(1).collect()[0]
arr = np.frombuffer(row["data"], dtype=np.uint8).reshape(IMG_SIZE, IMG_SIZE, 3)
print(f"1 registro          : label={row['label']} | shape={arr.shape} | dtype={arr.dtype} "
      f"| range=[{arr.min()}, {arr.max()}] (esperado 0-255)")

sz = subprocess.run(["du", "-sh", OUT_DIR], capture_output=True, text=True).stdout.split()[0]
n_files = len(list(Path(OUT_DIR).glob("*.parquet"))
print(f"Tamanho em disco      : {sz} em {n_files} arquivos parquet")
print(f"Bytes por imagem       : {IMG_SIZE*IMG_SIZE*3:,} (uint8)")
```

```
Registros no parquet : 70,433
Schema                : struct<label:int,data:binary>
1 registro            : label=1 | shape=(224, 224, 3) | dtype=uint8 | range=[1, 255] (esperado
0-255)
Tamanho em disco      : 9.4G em 35 arquivos parquet
Bytes por imagem       : 150,528 (uint8)
```

```
spark.stop()
print("Spark encerrado. Parquet uint8 pronto p/ o notebook 04 (benchmark do Pipeline B).")
```

```
Spark encerrado. Parquet uint8 pronto p/ o notebook 04 (benchmark do Pipeline B).
```

04 - Benchmark do Pipeline B: ingestão via .parquet (uint8)

Objetivo: rodar o **mesmo** pipeline de inferência do nb 02, mas lendo os tensores **prontos** do parquet uint8 (gerado no nb 03) em vez de decodificar `.jpg` em tempo real. Comparar os dois e mostrar o **GPU Starvation resolvido** + o ganho de throughput.

Igual ao nb 02 (de propósito, p/ comparação justa): MobileNetV2 forward-only + FP16, `ResourceMonitor` (CPU/RAM via cgroup), `N_ROUNDS` + warmup, tetos CPU/GPU, stats e gráficos.

Decisões de arquitetura (comentadas no código):

1. **Fonte do dado = parquet uint8**, lido com `pyarrow` e ligado ao `tf.data` via `from_generator` + `interleave` (leitura paralela) + `prefetch`. **Não** usamos `tensorflow-io` (leitor nativo de parquet): ele arrasta um `tensorflow` que sobrescreveria o TF otimizado da NVIDIA (NGC/Blackwell) e quebraria a GPU. E como gravamos bytes uint8, qualquer leitor teria que desserializar do mesmo jeito → `pyarrow` não perde em "limpeza".
2. **Preprocess na GPU:** o parquet guarda pixel cru; o `x/127.5 - 1` do MobileNet roda dentro do `infer` (na GPU). A CPU só lê o byte e transfere — e com metade dos bytes (uint8 = 1 vs fp16 = 2).
3. **Cache quente do SO:** o parquet (9,4 GB) cabe quase todo na RAM (16 GB) → leitura quente após a 1ª rodada. É justo: o jpg re-decodifica a cada rodada; o parquet lê quente — a vantagem que queremos mostrar. Disco não é gargalo (34k img/s no nb 02).
4. **N_IMAGES** : começa lendo TODAS as imagens do parquet (`None`); troque por um inteiro p/ reduzir.

Compara com `jpg_forward_stats.csv` (nb 02). Roda no contêiner (2 vCPU / 16 GB).

```

import os
import time
import threading
import glob
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import psutil
import pynvml
import pyarrow.parquet as pq
import tensorflow as tf

# FP16 (mesma config do nb 02). DEVE vir antes de construir o modelo.
tf.keras.mixed_precision.set_global_policy("mixed_float16")

PARQUET_DIR = "/tf/data/optimized"      # saída do nb 03 (uint8 224x224x3)
RESULTS_DIR = Path("/tf/data/results")
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

IMG_SIZE = (224, 224)
BATCH_SIZE = 256
N_IMAGES = None      # None = TODAS as imagens do parquet; um inteiro reduz (controle rapido)
N_PASSES = 1         # passadas por rodada (suba se reduzir muito o N_IMAGES)
N_ROUNDS = 3
SAMPLE_EVERY = 1.0
CYCLE = 2            # arquivos parquet lidos em paralelo (casa com os 2 vCPU)

for gpu in tf.config.list_physical_devices("GPU"):
    try:
        tf.config.experimental.set_memory_growth(gpu, True)
    except Exception as e:
        print("memory_growth:", e)

print("TensorFlow:", tf.__version__, "| policy:", tf.keras.mixed_precision.global_policy().name)

```

```

2026-07-01 05:46:46.138874: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You
may see slightly different numerical results due to floating-point round-off errors from different
computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-07-01 05:46:46.146450: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:485] Unable to
register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been
registered
2026-07-01 05:46:46.154282: E external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:8473] Unable
to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been
registered
2026-07-01 05:46:46.157075: E external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1471] Unable
to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already
been registered
2026-07-01 05:46:46.163808: I tensorflow/core/platform/cpu_feature_guard.cc:211] This TensorFlow
binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX, in other operations, rebuild
TensorFlow with the appropriate compiler flags.

```

```

INFO:tensorflow:Mixed precision compatibility check (mixed_float16): OK
Your GPU will likely run quickly with dtype policy mixed_float16 as it has compute capability of at
least 7.0. Your GPU: NVIDIA GeForce RTX 5060, compute capability 12.0
TensorFlow: 2.17.0 | policy: mixed_float16

```

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1782884807.997458    1362 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782884808.024391    1362 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782884808.025581    1362 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782884808.027148    1362 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
```

```
# Inicializa o pynvml p/ monitorar a GPU (igual ao nb 02).
try:
    pynvml.nvmlInit()
    GPU_HANDLE = pynvml.nvmlDeviceGetHandleByIndex(0)
    name = pynvml.nvmlDeviceGetName(GPU_HANDLE)
    print("GPU detectada:", name.decode() if isinstance(name, bytes) else name)
except Exception as e:
    GPU_HANDLE = None
    print("pynvml indisponivel, GPU nao sera monitorada:", e)
```

```
GPU detectada: NVIDIA GeForce RTX 5060
```



```

# -----
# Ingestao do parquet -> tf.data via PYARROW (nao tensorflow-io).
# -----
# Por que pyarrow e nao tf-io: o `tensorflow-io` tem leitor de parquet nativo, MAS arrasta uma
# dependencia `tensorflow` que sobrescreveria o TF otimizado da NVIDIA (NGC, Blackwell/sm_120)
# -> a RTX 5060 pararia de funcionar. Alem disso, como gravamos a imagem como BYTES uint8, qualquer
# leitor teria que desserializar (`frombuffer`) do mesmo jeito. Ponte via `from_generator` +
# `interleave` (le varios arquivos em paralelo) + `prefetch` -> a CPU entrega milhares de img/s.

PARQUET_FILES = sorted(glob.glob(f"{PARQUET_DIR}/*.parquet"))
if not PARQUET_FILES:
    raise FileNotFoundError(f"Nenhum .parquet em {PARQUET_DIR}. Rode o 03_spark_etl.ipynb antes.")
print(f"Arquivos parquet: {len(PARQUET_FILES)}")

def _parquet_gen(path):
    # Roda numa thread do tf.data; le 1 arquivo em lotes (o Arrow libera o GIL no I/O/deserialize,
    # entao o `interleave` paraleliza de verdade).
    if isinstance(path, bytes):
        path = path.decode()
    pf = pq.ParquetFile(path)
    for b in pf.iter_batches(batch_size=1024, columns=["label", "data"]):
        labels = b.column("label").to_numpy(zero_copy_only=False)
        buffers = b.column("data").to_pylist()
        for lbl, buf in zip(labels, buffers):
            # bytes uint8 -> tensor (224,224,3). SEM decode de JPEG -> esse e o ganho do Pipeline B.
            yield (np.frombuffer(buf, dtype=np.uint8).reshape(224, 224, 3),
                   np.float32(lbl))

def build_parquet_dataset(batch_size, n_images=None):
    ds = tf.data.Dataset.from_tensor_slices(PARQUET_FILES)
    ds = ds.interleave(
        lambda p: tf.data.Dataset.from_generator(
            _parquet_gen, args=(p,),
            output_signature=(
                tf.TensorSpec(shape=(224, 224, 3), dtype=tf.uint8),
                tf.TensorSpec(shape=(), dtype=tf.float32),
            ),
        ),
        cycle_length=CYCLE, # arquivos lidos em paralelo
        num_parallel_calls=tf.data.AUTOTUNE,
        deterministic=False,
    )
    if n_images is not None:
        ds = ds.take(n_images) # controle do tamanho (subset)
    ds = ds.batch(batch_size)
    ds = ds.prefetch(tf.data.AUTOTUNE) # sobrepeleitura (CPU) e inferencia (GPU)
    return ds

```

Arquivos parquet: 35

```
# Mesmo modelo do nb 02: MobileNetV2 forward-only + FP16.
def build_model():
    base = tf.keras.applications.MobileNetV2(
        weights="imagenet", include_top=False,
        input_shape=(*IMG_SIZE, 3), pooling="avg",
    )
    out = tf.keras.layers.Dense(1, activation="sigmoid", dtype="float32")(base.output)
    return tf.keras.Model(base.input, out)

model = build_model()

@tf.function
def infer(x_uint8):
    # PREPROCESS NA GPU: o parquet entrega pixel cru uint8; o cast + escala ([-1,1]) do MobileNet
    # roda aqui (na GPU). Assim a CPU só leu o byte e transferiu (uint8 = 1 byte, metade do fp16).
    x = tf.cast(x_uint8, tf.float16) / 127.5 - 1.0
    return model(x, training=False)

print("Modelo pronto (MobileNetV2 + FP16, preprocess na GPU). Params:", f"{model.count_params():,}")
```

```
I0000 00:00:1782884808.100718    1362 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782884808.102900    1362 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782884808.104077    1362 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782884808.176249    1362 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782884808.177220    1362 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
I0000 00:00:1782884808.178019    1362 cuda_executor.cc:1015] successful NUMA node read from SysFS
had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See
more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-
L355
2026-07-01 05:46:48.178774: I tensorflow/core/common_runtime/gpu/gpu_device.cc:2021] Created
device /job:localhost/replica:0/task:0/device:GPU:0 with 2923 MB memory: -> device: 0, name: NVIDIA
GeForce RTX 5060, pci bus id: 0000:01:00.0, compute capability: 12.0
```

```
Modelo pronto (MobileNetV2 + FP16, preprocess na GPU). Params: 2,259,265
```

```

# -----
# Monitor de recursos (IDENTICO ao nb 02): CPU/RAM do CONTEINER via cgroup + GPU via pynvml.
# -----
def detect_container_cpu_quota():
    try:
        with open("/sys/fs/cgroup/cpu.max") as f:
            q, p = f.read().strip().split()
            if q != "max":
                return float(q) / float(p)
    except (FileNotFoundError, OSError, ValueError):
        pass
    try:
        with open("/sys/fs/cgroup/cpu/cpu.cfs_quota_us") as f:
            q = int(f.read().strip())
        with open("/sys/fs/cgroup/cpu/cpu.cfs_period_us") as f:
            p = int(f.read().strip())
        if q > 0:
            return q / p
    except (FileNotFoundError, OSError, ValueError):
        pass
    try:
        return float(len(os.sched_getaffinity(0)))
    except (AttributeError, OSError):
        return float(os.cpu_count() or 1)

def read_container_mem():
    try:
        with open("/sys/fs/cgroup/memory.current") as f:
            used = int(f.read().strip())
        with open("/sys/fs/cgroup/memory.max") as f:
            lim = f.read().strip()
            limit = None if lim == "max" else int(lim)
            return used / 1024 ** 3, (limit / 1024 ** 3 if limit else float("nan"))
    except (FileNotFoundError, OSError, ValueError):
        pass
    try:
        with open("/sys/fs/cgroup/memory/memory.usage_in_bytes") as f:
            used = int(f.read().strip())
        with open("/sys/fs/cgroup/memory/memory.limit_in_bytes") as f:
            limit = int(f.read().strip())
            limit = limit if limit < (1 << 62) else None
            return used / 1024 ** 3, (limit / 1024 ** 3 if limit else float("nan"))
    except (FileNotFoundError, OSError, ValueError):
        return float("nan"), float("nan")

N_CONTAINER_CPUS = detect_container_cpu_quota()
_, CONTAINER_MEM_LIMIT_GB = read_container_mem()
print(f"vCPUs (cgroup): {N_CONTAINER_CPUS} | RAM limite (cgroup): {CONTAINER_MEM_LIMIT_GB:.1f} GB")

class ResourceMonitor(threading.Thread):
    def __init__(self, gpu_handle, interval=1.0, n_container_cpus=N_CONTAINER_CPUS):
        super().__init__(daemon=True)
        self.handle = gpu_handle
        self.interval = interval
        self.n_container_cpus = max(float(n_container_cpus), 1e-6)
        self._stop_event = threading.Event()
        self.samples = []
        self.proc = psutil.Process()

    def run(self):
        psutil.cpu_percent(None); self.proc.cpu_percent(None)
        t_start = time.time()
        while not self._stop_event.is_set():
            cpu_host = psutil.cpu_percent(None)
            proc_cpu_raw = self.proc.cpu_percent(None)
            cpu_container = min(proc_cpu_raw / self.n_container_cpus, 100.0)
            cont_used_gb, _ = read_container_mem()
            proc_rss_gb = self.proc.memory_info().rss / (1024 ** 3)
            ram_used_host_gb = psutil.virtual_memory().used / (1024 ** 3)
            gpu_util, gpu_mem_gb = np.nan, np.nan

```

```

    if self.handle is not None:
        try:
            gpu_util = float(pynvml.nvmlDeviceGetUtilizationRates(self.handle).gpu)
            gpu_mem_gb = pynvml.nvmlDeviceGetMemoryInfo(self.handle).used / (1024 ** 3)
        except Exception:
            pass
    self.samples.append({
        "t_rel": time.time() - t_start, "cpu_pct": cpu_container, "cpu_pct_host": cpu_host,
        "proc_cpu_raw": proc_cpu_raw, "cont_mem_used_gb": cont_used_gb,
        "proc_rss_gb": proc_rss_gb, "ram_used_host_gb": ram_used_host_gb,
        "gpu_util_pct": gpu_util, "gpu_mem_gb": gpu_mem_gb,
    })
    self._stop_event.wait(self.interval)

def stop(self):
    self._stop_event.set(); self.join()
    return pd.DataFrame(self.samples)

```

vCPUs (cgroup): 2.0 | RAM limite (cgroup): 16.0 GB

```

round_summaries, raw_samples = [], []

# Warmup FORA do cronometro (traca o infer + esquentar cache/pipeline) -> rodada 1 nao infla o desvio.
_ds_warm = build_parquet_dataset(BATCH_SIZE, N_IMAGES)
for x, _ in _ds_warm.take(5):
    infer(x)

for r in range(1, N_ROUNDS + 1):
    print(f"\n===== Rodada {r}/{N_ROUNDS} =====")
    ds = build_parquet_dataset(BATCH_SIZE, N_IMAGES) # le o parquet (cache quente apos a rodada 1)

    monitor = ResourceMonitor(GPU_HANDLE, interval=SAMPLE_EVERY)
    monitor.start()
    n_images = 0
    t0 = time.time()
    for _ in range(N_PASSES):
        for x, y in ds:
            infer(x) # forward (preprocess na GPU)
            n_images += int(x.shape[0])
    elapsed = time.time() - t0
    df_s = monitor.stop(); df_s["round"] = r; raw_samples.append(df_s)

    tp = n_images / elapsed
    round_summaries.append({
        "round": r, "n_images": n_images, "round_time_s": elapsed, "throughput_img_s": tp,
        "cpu_mean_pct": df_s["cpu_pct"].mean(), "cpu_max_pct": df_s["cpu_pct"].max(),
        "cont_mem_peak_gb": df_s["cont_mem_used_gb"].max(), "proc_rss_peak_gb":
df_s["proc_rss_gb"].max(),
        "ram_host_peak_gb": df_s["ram_used_host_gb"].max(),
        "gpu_util_mean_pct": df_s["gpu_util_pct"].mean(), "gpu_util_max_pct":
df_s["gpu_util_pct"].max(),
        "gpu_mem_peak_gb": df_s["gpu_mem_gb"].max(),
    })
    s = round_summaries[-1]
    print(f"  imagens={n_images:} | tempo={elapsed:.1f}s | throughput={tp:.1f} img/s | "
          f"CPU={s['cpu_mean_pct']:.0f}% | GPU={s['gpu_util_mean_pct']:.0f}%")

summary_df = pd.DataFrame(round_summaries)
raw_df = pd.concat(raw_samples, ignore_index=True)
summary_df

```

```
2026-07-01 05:46:50.258892: E tensorflow/core/util/util.cc:131] oneDNN supports DT_HALF only on
platforms with AVX-512. Falling back to the default Eigen-based implementation if present.
2026-07-01 05:46:50.296199: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:531] Loaded
cuDNN version 90701
W0000 00:00:1782884810.329387 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.330364 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.331362 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.332501 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.336706 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.340194 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.344306 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.353295 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.362079 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.368716 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.390640 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.394860 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.400145 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.404589 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.408686 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.416444 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.422212 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.429037 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.436891 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.444301 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.451522 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.460578 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.467491 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.476879 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.485841 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.495879 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.506393 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.517429 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.528599 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.538562 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.555717 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.592224 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.617991 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.620946 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884810.622488 1425 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
```

[illegible]

[illegible]

[illegible]

accuracy will be reduced
W0000 00:00:1782884812.001979
accuracy will be reduced
W0000 00:00:1782884812.007196
accuracy will be reduced
W0000 00:00:1782884812.013966
accuracy will be reduced
W0000 00:00:1782884812.017768
accuracy will be reduced
W0000 00:00:1782884812.021980
accuracy will be reduced
W0000 00:00:1782884812.027388
accuracy will be reduced
W0000 00:00:1782884812.031773
accuracy will be reduced
W0000 00:00:1782884812.035836
accuracy will be reduced
W0000 00:00:1782884812.043846
accuracy will be reduced
W0000 00:00:1782884812.049585
accuracy will be reduced
W0000 00:00:1782884812.056208
accuracy will be reduced
W0000 00:00:1782884812.064295
accuracy will be reduced
W0000 00:00:1782884812.071700
accuracy will be reduced
W0000 00:00:1782884812.079051
accuracy will be reduced
W0000 00:00:1782884812.087985
accuracy will be reduced
W0000 00:00:1782884812.095205
accuracy will be reduced
W0000 00:00:1782884812.104419
accuracy will be reduced
W0000 00:00:1782884812.113645
accuracy will be reduced
W0000 00:00:1782884812.123386
accuracy will be reduced
W0000 00:00:1782884812.133592
accuracy will be reduced
W0000 00:00:1782884812.144238
accuracy will be reduced
W0000 00:00:1782884812.155164
accuracy will be reduced
W0000 00:00:1782884812.165250
accuracy will be reduced
W0000 00:00:1782884812.180127
accuracy will be reduced
W0000 00:00:1782884812.204817
accuracy will be reduced
W0000 00:00:1782884812.227859
accuracy will be reduced
W0000 00:00:1782884812.231390
accuracy will be reduced
W0000 00:00:1782884812.232814
accuracy will be reduced
W0000 00:00:1782884812.233970
accuracy will be reduced
W0000 00:00:1782884812.235394
accuracy will be reduced
W0000 00:00:1782884812.236678
accuracy will be reduced
W0000 00:00:1782884812.248380
accuracy will be reduced
W0000 00:00:1782884812.284057
accuracy will be reduced
W0000 00:00:1782884812.323457
accuracy will be reduced
W0000 00:00:1782884812.391952
accuracy will be reduced
W0000 00:00:1782884812.468563
accuracy will be reduced
W0000 00:00:1782884812.527811

[illegible]

accuracy will be reduced
W0000 00:00:1782884812.547623
accuracy will be reduced
W0000 00:00:1782884812.549757
accuracy will be reduced
W0000 00:00:1782884812.551797
accuracy will be reduced
W0000 00:00:1782884812.554471
accuracy will be reduced
W0000 00:00:1782884812.556531
accuracy will be reduced
W0000 00:00:1782884812.583550
accuracy will be reduced
W0000 00:00:1782884812.658404
accuracy will be reduced
W0000 00:00:1782884812.732459
accuracy will be reduced
W0000 00:00:1782884812.808454
accuracy will be reduced
W0000 00:00:1782884812.874107
accuracy will be reduced
W0000 00:00:1782884813.009886
accuracy will be reduced
W0000 00:00:1782884813.018817
accuracy will be reduced
W0000 00:00:1782884813.020215
accuracy will be reduced
W0000 00:00:1782884813.021815
accuracy will be reduced
W0000 00:00:1782884813.023873
accuracy will be reduced
W0000 00:00:1782884813.025326
accuracy will be reduced
W0000 00:00:1782884813.056944
accuracy will be reduced
W0000 00:00:1782884813.160628
accuracy will be reduced
W0000 00:00:1782884813.170791
accuracy will be reduced
W0000 00:00:1782884813.171575
accuracy will be reduced
W0000 00:00:1782884813.172542
accuracy will be reduced
W0000 00:00:1782884813.173512
accuracy will be reduced
W0000 00:00:1782884813.174261
accuracy will be reduced
W0000 00:00:1782884813.182855
accuracy will be reduced
W0000 00:00:1782884813.215704
accuracy will be reduced
W0000 00:00:1782884813.224496
accuracy will be reduced
W0000 00:00:1782884813.225004
accuracy will be reduced
W0000 00:00:1782884813.225278
accuracy will be reduced
W0000 00:00:1782884813.225556
accuracy will be reduced
W0000 00:00:1782884813.225865
accuracy will be reduced
W0000 00:00:1782884813.227630
accuracy will be reduced
W0000 00:00:1782884813.240203
accuracy will be reduced
W0000 00:00:1782884813.251345
accuracy will be reduced
W0000 00:00:1782884813.251693
accuracy will be reduced
W0000 00:00:1782884813.251883
accuracy will be reduced
W0000 00:00:1782884813.252100
accuracy will be reduced
W0000 00:00:1782884813.252300

[illegible]

accuracy will be reduced
W0000 00:00:1782884813.253618
accuracy will be reduced
W0000 00:00:1782884813.265790
accuracy will be reduced
W0000 00:00:1782884813.267714
accuracy will be reduced
W0000 00:00:1782884813.268519
accuracy will be reduced
W0000 00:00:1782884813.269568
accuracy will be reduced
W0000 00:00:1782884813.271365
accuracy will be reduced
W0000 00:00:1782884813.275160
accuracy will be reduced
W0000 00:00:1782884813.279070
accuracy will be reduced
W0000 00:00:1782884813.285406
accuracy will be reduced
W0000 00:00:1782884813.290797
accuracy will be reduced
W0000 00:00:1782884813.297823
accuracy will be reduced
W0000 00:00:1782884813.301592
accuracy will be reduced
W0000 00:00:1782884813.305314
accuracy will be reduced
W0000 00:00:1782884813.311170
accuracy will be reduced
W0000 00:00:1782884813.315595
accuracy will be reduced
W0000 00:00:1782884813.319356
accuracy will be reduced
W0000 00:00:1782884813.327446
accuracy will be reduced
W0000 00:00:1782884813.333200
accuracy will be reduced
W0000 00:00:1782884813.339341
accuracy will be reduced
W0000 00:00:1782884813.347774
accuracy will be reduced
W0000 00:00:1782884813.354666
accuracy will be reduced
W0000 00:00:1782884813.362410
accuracy will be reduced
W0000 00:00:1782884813.370829
accuracy will be reduced
W0000 00:00:1782884813.378210
accuracy will be reduced
W0000 00:00:1782884813.387181
accuracy will be reduced
W0000 00:00:1782884813.396649
accuracy will be reduced
W0000 00:00:1782884813.406372
accuracy will be reduced
W0000 00:00:1782884813.416863
accuracy will be reduced
W0000 00:00:1782884813.427815
accuracy will be reduced
W0000 00:00:1782884813.438667
accuracy will be reduced
W0000 00:00:1782884813.449298
accuracy will be reduced
W0000 00:00:1782884813.464807
accuracy will be reduced
W0000 00:00:1782884813.490070
accuracy will be reduced
W0000 00:00:1782884813.513624
accuracy will be reduced
W0000 00:00:1782884813.517145
accuracy will be reduced
W0000 00:00:1782884813.518575
accuracy will be reduced
W0000 00:00:1782884813.519730

[illegible]

accuracy will be reduced
W0000 00:00:1782884813.520935
accuracy will be reduced
W0000 00:00:1782884813.522117
accuracy will be reduced
W0000 00:00:1782884813.534236
accuracy will be reduced
W0000 00:00:1782884813.569491
accuracy will be reduced
W0000 00:00:1782884813.608548
accuracy will be reduced
W0000 00:00:1782884813.676464
accuracy will be reduced
W0000 00:00:1782884813.751783
accuracy will be reduced
W0000 00:00:1782884813.811211
accuracy will be reduced
W0000 00:00:1782884813.838563
accuracy will be reduced
W0000 00:00:1782884813.839994
accuracy will be reduced
W0000 00:00:1782884813.841712
accuracy will be reduced
W0000 00:00:1782884813.843632
accuracy will be reduced
W0000 00:00:1782884813.845173
accuracy will be reduced
W0000 00:00:1782884813.876950
accuracy will be reduced
W0000 00:00:1782884813.980814
accuracy will be reduced
W0000 00:00:1782884814.011170
accuracy will be reduced
W0000 00:00:1782884814.011633
accuracy will be reduced
W0000 00:00:1782884814.011829
accuracy will be reduced
W0000 00:00:1782884814.012050
accuracy will be reduced
W0000 00:00:1782884814.012257
accuracy will be reduced
W0000 00:00:1782884814.013576
accuracy will be reduced
W0000 00:00:1782884814.025029
accuracy will be reduced
W0000 00:00:1782884814.027237
accuracy will be reduced
W0000 00:00:1782884814.028218
accuracy will be reduced
W0000 00:00:1782884814.029344
accuracy will be reduced
W0000 00:00:1782884814.031151
accuracy will be reduced
W0000 00:00:1782884814.034679
accuracy will be reduced
W0000 00:00:1782884814.038334
accuracy will be reduced
W0000 00:00:1782884814.045216
accuracy will be reduced
W0000 00:00:1782884814.050445
accuracy will be reduced
W0000 00:00:1782884814.057077
accuracy will be reduced
W0000 00:00:1782884814.061110
accuracy will be reduced
W0000 00:00:1782884814.065286
accuracy will be reduced
W0000 00:00:1782884814.070480
accuracy will be reduced
W0000 00:00:1782884814.074884
accuracy will be reduced
W0000 00:00:1782884814.079127
accuracy will be reduced
W0000 00:00:1782884814.086613

[illegible]

accuracy will be reduced
W0000 00:00:1782884814.092354
accuracy will be reduced
W0000 00:00:1782884814.099212
accuracy will be reduced
W0000 00:00:1782884814.106981
accuracy will be reduced
W0000 00:00:1782884814.114584
accuracy will be reduced
W0000 00:00:1782884814.121674
accuracy will be reduced
W0000 00:00:1782884814.130911
accuracy will be reduced
W0000 00:00:1782884814.137673
accuracy will be reduced
W0000 00:00:1782884814.147122
accuracy will be reduced
W0000 00:00:1782884814.155795
accuracy will be reduced
W0000 00:00:1782884814.165665
accuracy will be reduced
W0000 00:00:1782884814.175614
accuracy will be reduced
W0000 00:00:1782884814.186333
accuracy will be reduced
W0000 00:00:1782884814.197259
accuracy will be reduced
W0000 00:00:1782884814.207219
accuracy will be reduced
W0000 00:00:1782884814.222060
accuracy will be reduced
W0000 00:00:1782884814.246719
accuracy will be reduced
W0000 00:00:1782884814.269451
accuracy will be reduced
W0000 00:00:1782884814.272973
accuracy will be reduced
W0000 00:00:1782884814.274418
accuracy will be reduced
W0000 00:00:1782884814.275576
accuracy will be reduced
W0000 00:00:1782884814.276958
accuracy will be reduced
W0000 00:00:1782884814.278318
accuracy will be reduced
W0000 00:00:1782884814.289916
accuracy will be reduced
W0000 00:00:1782884814.325170
accuracy will be reduced
W0000 00:00:1782884814.364543
accuracy will be reduced
W0000 00:00:1782884814.432731
accuracy will be reduced
W0000 00:00:1782884814.507639
accuracy will be reduced
W0000 00:00:1782884814.566094
accuracy will be reduced
W0000 00:00:1782884814.592546
accuracy will be reduced
W0000 00:00:1782884814.594071
accuracy will be reduced
W0000 00:00:1782884814.595838
accuracy will be reduced
W0000 00:00:1782884814.597777
accuracy will be reduced
W0000 00:00:1782884814.599109
accuracy will be reduced
W0000 00:00:1782884814.629849
accuracy will be reduced
W0000 00:00:1782884814.734286
accuracy will be reduced
W0000 00:00:1782884814.764457
accuracy will be reduced
W0000 00:00:1782884814.765000

[illegible]

accuracy will be reduced
W0000 00:00:1782884814.765324
accuracy will be reduced
W0000 00:00:1782884814.765542
accuracy will be reduced
W0000 00:00:1782884814.765746
accuracy will be reduced
W0000 00:00:1782884814.767065
accuracy will be reduced
W0000 00:00:1782884814.778544
accuracy will be reduced
W0000 00:00:1782884814.780561
accuracy will be reduced
W0000 00:00:1782884814.781359
accuracy will be reduced
W0000 00:00:1782884814.782599
accuracy will be reduced
W0000 00:00:1782884814.784439
accuracy will be reduced
W0000 00:00:1782884814.787898
accuracy will be reduced
W0000 00:00:1782884814.791553
accuracy will be reduced
W0000 00:00:1782884814.798456
accuracy will be reduced
W0000 00:00:1782884814.803690
accuracy will be reduced
W0000 00:00:1782884814.810570
accuracy will be reduced
W0000 00:00:1782884814.814350
accuracy will be reduced
W0000 00:00:1782884814.818399
accuracy will be reduced
W0000 00:00:1782884814.823765
accuracy will be reduced
W0000 00:00:1782884814.828444
accuracy will be reduced
W0000 00:00:1782884814.832221
accuracy will be reduced
W0000 00:00:1782884814.840003
accuracy will be reduced
W0000 00:00:1782884814.845997
accuracy will be reduced
W0000 00:00:1782884814.852657
accuracy will be reduced
W0000 00:00:1782884814.860688
accuracy will be reduced
W0000 00:00:1782884814.868073
accuracy will be reduced
W0000 00:00:1782884814.875230
accuracy will be reduced
W0000 00:00:1782884814.884370
accuracy will be reduced
W0000 00:00:1782884814.891167
accuracy will be reduced
W0000 00:00:1782884814.900573
accuracy will be reduced
W0000 00:00:1782884814.909632
accuracy will be reduced
W0000 00:00:1782884814.919363
accuracy will be reduced
W0000 00:00:1782884814.929528
accuracy will be reduced
W0000 00:00:1782884814.940093
accuracy will be reduced
W0000 00:00:1782884814.951088
accuracy will be reduced
W0000 00:00:1782884814.961284
accuracy will be reduced
W0000 00:00:1782884814.976254
accuracy will be reduced
W0000 00:00:1782884815.001551
accuracy will be reduced

[illegible]

2026-07-01 05:46:55.025291: I tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

==== Rodada 1/3 ====

[illegible]

[illegible]

[illegible]

```
W0000 00:00:1782884836.705915 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.705951 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.705996 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.706630 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.706663 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.706706 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.706746 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.706789 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.706836 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.707819 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.707861 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.707926 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.707979 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.708037 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.708100 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.709128 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.709157 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.709209 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.709246 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.709282 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.709321 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.709985 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.710022 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.710080 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.710124 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.710173 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.710226 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
W0000 00:00:1782884836.710490 1428 gpu_timer.cc:114] Skipping the delay kernel, measurement
accuracy will be reduced
2026-07-01 05:47:16.712608: I tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is
aborting with status: OUT_OF_RANGE: End of sequence
```

imagens=70,433 | tempo=21.7s | throughput=3249.7 img/s | CPU=86% | GPU=94%

===== Rodada 2/3 =====

[illegible]

W0000 00:00:1782884857.550345 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.555230 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.560324 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.568798 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.579620 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.581473 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.581713 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.581979 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.582250 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.582516 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.585896 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.595089 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.603486 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.613302 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.623938 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.624555 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.624717 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.624893 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.625066 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.625250 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.629594 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.630397 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.630488 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.630591 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.630703 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.630784 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.631891 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.633745 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.634099 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.634282 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.634444 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.634475 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement
W0000 00:00:1782884857.634515 accuracy will be reduced	1427 gpu_timer.cc:114] Skipping the delay kernel, measurement

imagens=70,433 | tempo=20.9s | throughput=3365.6 img/s | CPU=87% | GPU=92%

===== Rodada 3/3 =====

imagens=70,433 | tempo=21.4s | throughput=3289.3 img/s | CPU=85% | GPU=91%

2026-07-01 05:47:59.225347: I tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

	round	n_images	round_time_s	throughput_img_s	cpu_mean_pct	cpu_max_pct	cont_mem_peak_gb	pr
0	1	70433	21.673921	3249.665792	85.595455	95.95	15.979000	7.
1	2	70433	20.927145	3365.628708	86.716667	98.45	15.984081	8.
2	3	70433	21.412519	3289.337387	84.538636	95.45	15.998787	7.

```
# Tetos isolados (mesma logica do nb 02) -> quantifica o starvation deste pipeline.
x_fixed, _ = next(iter(build_parquet_dataset(BATCH_SIZE, N_IMAGES)))
infer(x_fixed) # warmup do teste de GPU
N_STEPS = max(30, 15000 // BATCH_SIZE)

cpu_ceils, gpu_ceils = [], []
for r in range(1, N_ROUNDS + 1):
    # TETO CPU: leitura+deserializacao do parquet, SEM modelo (produtor)
    ds_cpu = build_parquet_dataset(BATCH_SIZE, N_IMAGES)
    n, t0 = 0, time.time()
    for _ in range(N_PASSES):
        for x, y in ds_cpu:
            n += int(x.shape[0])
    cpu_ceils.append(n / (time.time() - t0))
    # TETO GPU: forward sobre 1 batch fixo uint8 (consumidor) -> deve bater ~o do nb 02 (mesmo modelo)
    n, t0 = 0, time.time()
    for _ in range(N_STEPS):
        infer(x_fixed)
        n += int(x_fixed.shape[0])
    gpu_ceils.append(n / (time.time() - t0))
    print(f" rodada {r}: teto CPU(leitura)={cpu_ceils[-1]:.0f} | teto GPU={gpu_ceils[-1]:.0f} img/s")

cpu_ceiling = float(np.mean(cpu_ceils)); cpu_ceiling_std = float(np.std(cpu_ceils, ddof=1)) if
N_ROUNDS > 1 else 0.0
gpu_ceiling = float(np.mean(gpu_ceils)); gpu_ceiling_std = float(np.std(gpu_ceils, ddof=1)) if
N_ROUNDS > 1 else 0.0
print(f"\nTeto CPU (leitura parquet): {cpu_ceiling:.0f} +/- {cpu_ceiling_std:.0f} img/s (produtor)")
print(f"Teto GPU (forward) : {gpu_ceiling:.0f} +/- {gpu_ceiling_std:.0f} img/s (consumidor)")
```

rodada 1: teto CPU(leitura)=6418 | teto GPU=3663 img/s
rodada 2: teto CPU(leitura)=6261 | teto GPU=3646 img/s
rodada 3: teto CPU(leitura)=6313 | teto GPU=3660 img/s

Teto CPU (leitura parquet): 6331 +/- 80 img/s (produtor)
Teto GPU (forward) : 3656 +/- 9 img/s (consumidor)

```

tp = summary_df["throughput_img_s"].values
tp_mean = float(tp.mean()); tp_std = float(tp.std(ddof=1)) if len(tp) > 1 else 0.0
gpu_waste_pct = 100.0 * (1 - tp_mean / gpu_ceiling) if gpu_ceiling else float("nan")

stats = {
    "pipeline": "parquet_uint8_forward",
    "n_rounds": N_ROUNDS, "batch_size": BATCH_SIZE,
    "n_images_round": int(summary_df["n_images"].iloc[0]),
    "throughput_mean_img_s": tp_mean, "throughput_std_img_s": tp_std,
    "cpu_ceiling_img_s": cpu_ceiling, "cpu_ceiling_std_img_s": cpu_ceiling_std,
    "gpu_ceiling_img_s": gpu_ceiling, "gpu_ceiling_std_img_s": gpu_ceiling_std,
    "gpu_waste_pct": gpu_waste_pct,
    "round_time_mean_s": float(summary_df["round_time_s"].mean()),
    "cpu_mean_pct": float(summary_df["cpu_mean_pct"].mean()),
    "gpu_util_mean_pct": float(summary_df["gpu_util_mean_pct"].mean()),
    "cont_mem_peak_gb": float(summary_df["cont_mem_peak_gb"].max()),
    "gpu_mem_peak_gb": float(summary_df["gpu_mem_peak_gb"].max()),
}
stats_df = pd.DataFrame([stats])
summary_df.to_csv(RESULTS_DIR / "parquet_forward_per_round.csv", index=False)
stats_df.to_csv(RESULTS_DIR / "parquet_forward_stats.csv", index=False)
raw_df.to_csv(RESULTS_DIR / "parquet_forward_raw_samples.csv", index=False)

print(f"Teto CPU (leitura) : {cpu_ceiling:.0f} +/- {cpu_ceiling_std:.0f} img/s (produtor)")
print(f"Teto GPU (forward) : {gpu_ceiling:.0f} +/- {gpu_ceiling_std:.0f} img/s (consumidor)")
print(f"Pipeline parquet : {tp_mean:.0f} +/- {tp_std:.0f} img/s")
print(f">> GPU desperdicada : {gpu_waste_pct:.0f}% (starvation)")
print(f"GPU util {stats['gpu_util_mean_pct']:.0f}% | CPU {stats['cpu_mean_pct']:.0f}% | "
      f"RAM {stats['cont_mem_peak_gb']:.2f}/{CONTAINER_MEM_LIMIT_GB:.1f} GB")
stats_df

```

```

Teto CPU (leitura) : 6331 +/- 80 img/s (produtor)
Teto GPU (forward) : 3656 +/- 9 img/s (consumidor)
Pipeline parquet : 3302 +/- 59 img/s
>> GPU desperdicada : 10% (starvation)
GPU util 92% | CPU 86% | RAM 16.00/16.0 GB

```

	pipeline	n_rounds	batch_size	n_images_round	throughput_mean_img_s	throughput_std_img_s
0	parquet_uint8_forward	3	256	70433	3301.543962	58.937253

Comparação final: **.jpg** (Pipeline A) vs **.parquet** (Pipeline B)

Carrega o **jpg_forward_stats.csv** (nb 02) e compara throughput, utilização e starvation — o resultado que prova a tese (decode movido pro ETL → CPU liberada → GPU alimentada).


```

jpg_csv = RESULTS_DIR / "jpg_forward_stats.csv"
if not jpg_csv.exists():
    print(f"[!] {jpg_csv} nao encontrado. Rode o nb 02 (jpg_forward) antes p/ a comparacao.")
else:
    jpg = pd.read_csv(jpg_csv).iloc[0]
    speedup = tp_mean / jpg["throughput_mean_img_s"]
    cmp = pd.DataFrame({
        "metrica": ["throughput (img/s)", "GPU util (%)", "CPU util (%)",
                    "teto CPU/leitura (img/s)", "GPU desperdicada (%)"],
        "jpg (A)": [jpg["throughput_mean_img_s"], jpg["gpu_util_mean_pct"], jpg["cpu_mean_pct"],
                    jpg["cpu_ceiling_img_s"], jpg["gpu_waste_pct"]],
        "parquet (B)": [tp_mean, stats["gpu_util_mean_pct"], stats["cpu_mean_pct"],
                        cpu_ceiling, gpu_waste_pct],
        "delta": [f"{speedup:.2f}x",
                  f"{stats['gpu_util_mean_pct'] - jpg['gpu_util_mean_pct']:+.0f} pp",
                  f"{stats['cpu_mean_pct'] - jpg['cpu_mean_pct']:+.0f} pp",
                  f"{cpu_ceiling / jpg['cpu_ceiling_img_s']:.2f}x",
                  f"{gpu_waste_pct - jpg['gpu_waste_pct']:+.0f} pp"],
    })
    print(cmp.to_string(index=False))
    cmp.to_csv(RESULTS_DIR / "comparacao_jpg_vs_parquet.csv", index=False)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
ax = axes[0]
ax.bar(["jpg (A)", "parquet (B)"], [jpg["throughput_mean_img_s"], tp_mean],
       yerr=[jpg["throughput_std_img_s"], tp_std], capsize=6,
       color=["#c0392b", "#27ae60"], alpha=0.85)
ax.set_title(f"Throughput de ingestao ({speedup:.2f}x)")
ax.set_ylabel("imagens / segundo")
for i, v in enumerate([jpg["throughput_mean_img_s"], tp_mean]):
    ax.text(i, v, f"{v:.0f}", ha="center", va="bottom")

ax = axes[1]
x = np.arange(2); w = 0.35
ax.bar(x - w/2, [jpg["cpu_mean_pct"], stats["cpu_mean_pct"]], w, label="CPU", color="#2980b9",
alpha=0.85)
ax.bar(x + w/2, [jpg["gpu_util_mean_pct"], stats["gpu_util_mean_pct"]], w, label="GPU",
color="#27ae60", alpha=0.85)
ax.set_xticks(x); ax.set_xticklabels(["jpg (A)", "parquet (B)"]); ax.set_ylim(0, 100)
ax.set_title("Utilizacao CPU vs GPU"); ax.set_ylabel("%"); ax.legend()
plt.tight_layout()
plt.savefig(RESULTS_DIR / "comparacao_jpg_vs_parquet.png", dpi=120, bbox_inches="tight")
plt.show()

```

	metrica	jpg (A)	parquet (B)	delta
throughput (img/s)		2432.889639	3301.543962	1.36x
GPU util (%)		76.629656	92.375180	+16 pp
CPU util (%)		92.181987	85.616919	-7 pp
teto CPU/leitura (img/s)		2760.659282	6330.842059	2.29x
GPU desperdicada (%)		27.256097	9.698885	-18 pp

